

Untitled7

March 23, 2025

```
[176]: import pandas as pd
import numpy as np
```

```
[177]: df = pd.read_csv("laptop_data.csv")
df.head()
```

```
[177]: Unnamed: 0  Company  TypeName  Inches  ScreenResolution \
0          0   Apple  Ultrabook   13.3  IPS Panel Retina Display 2560x1600
1          1   Apple  Ultrabook   13.3                1440x900
2          2     HP   Notebook   15.6                Full HD 1920x1080
3          3   Apple  Ultrabook   15.4  IPS Panel Retina Display 2880x1800
4          4   Apple  Ultrabook   13.3  IPS Panel Retina Display 2560x1600
```

```
          Cpu  Ram  Memory \
0  Intel Core i5 2.3GHz  8GB  128GB SSD
1  Intel Core i5 1.8GHz  8GB 128GB Flash Storage
2  Intel Core i5 7200U 2.5GHz  8GB  256GB SSD
3  Intel Core i7 2.7GHz 16GB  512GB SSD
4  Intel Core i5 3.1GHz  8GB  256GB SSD
```

```
          Gpu  OpSys  Weight  Price
0  Intel Iris Plus Graphics 640  macOS  1.37kg  71378.6832
1  Intel HD Graphics 6000  macOS  1.34kg  47895.5232
2  Intel HD Graphics 620  No OS  1.86kg  30636.0000
3  AMD Radeon Pro 455  macOS  1.83kg  135195.3360
4  Intel Iris Plus Graphics 650  macOS  1.37kg  96095.8080
```

```
[178]: df.shape
```

```
[178]: (1303, 12)
```

```
[179]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1303 entries, 0 to 1302
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            1303 non-null  int64
```

```

1   Company      1303 non-null  object
2   TypeName     1303 non-null  object
3   Inches       1303 non-null  float64
4   ScreenResolution 1303 non-null  object
5   Cpu          1303 non-null  object
6   Ram          1303 non-null  object
7   Memory       1303 non-null  object
8   Gpu          1303 non-null  object
9   OpSys        1303 non-null  object
10  Weight       1303 non-null  object
11  Price        1303 non-null  float64

```

dtypes: float64(2), int64(1), object(9)

memory usage: 122.3+ KB

```
[180]: df.isnull().sum()
```

```

[180]: Unnamed: 0      0
      Company      0
      TypeName      0
      Inches      0
      ScreenResolution 0
      Cpu          0
      Ram          0
      Memory      0
      Gpu          0
      OpSys        0
      Weight      0
      Price       0
      dtype: int64

```

```
[181]: df.duplicated().sum()
```

```
[181]: 0
```

```
[182]: df.drop(columns=["Unnamed: 0"],inplace=True)
```

```
[183]: df.head()
```

```

[183]:   Company  TypeName  Inches  IPS Panel Retina Display  ScreenResolution \
0   Apple  Ultrabook   13.3  IPS Panel Retina Display  2560x1600
1   Apple  Ultrabook   13.3                        1440x900
2    HP    Notebook   15.6                        Full HD 1920x1080
3   Apple  Ultrabook   15.4  IPS Panel Retina Display  2880x1800
4   Apple  Ultrabook   13.3  IPS Panel Retina Display  2560x1600

      Cpu      Ram      Memory \
0  Intel Core i5 2.3GHz  8GB      128GB SSD
1  Intel Core i5 1.8GHz  8GB  128GB Flash Storage

```

2	Intel Core i5 7200U 2.5GHz	8GB	256GB SSD
3	Intel Core i7 2.7GHz	16GB	512GB SSD
4	Intel Core i5 3.1GHz	8GB	256GB SSD

		Gpu	OpSys	Weight	Price
0	Intel Iris Plus Graphics	640	macOS	1.37kg	71378.6832
1	Intel HD Graphics	6000	macOS	1.34kg	47895.5232
2	Intel HD Graphics	620	No OS	1.86kg	30636.0000
3	AMD Radeon Pro	455	macOS	1.83kg	135195.3360
4	Intel Iris Plus Graphics	650	macOS	1.37kg	96095.8080

```
[184]: df['Ram']=df['Ram'].str.replace('GB','')
df['Weight']=df['Weight'].str.replace('kg','')
```

```
[185]: df.head()
```

```
[185]: Company   TypeName  Inches      ScreenResolution \
0   Apple  Ultrabook   13.3  IPS Panel Retina Display 2560x1600
1   Apple  Ultrabook   13.3                1440x900
2    HP    Notebook   15.6                Full HD 1920x1080
3   Apple  Ultrabook   15.4  IPS Panel Retina Display 2880x1800
4   Apple  Ultrabook   13.3  IPS Panel Retina Display 2560x1600
```

		Cpu	Ram	Memory \
0	Intel Core i5 2.3GHz	8	128GB SSD	
1	Intel Core i5 1.8GHz	8	128GB Flash Storage	
2	Intel Core i5 7200U 2.5GHz	8	256GB SSD	
3	Intel Core i7 2.7GHz	16	512GB SSD	
4	Intel Core i5 3.1GHz	8	256GB SSD	

		Gpu	OpSys	Weight	Price
0	Intel Iris Plus Graphics	640	macOS	1.37	71378.6832
1	Intel HD Graphics	6000	macOS	1.34	47895.5232
2	Intel HD Graphics	620	No OS	1.86	30636.0000
3	AMD Radeon Pro	455	macOS	1.83	135195.3360
4	Intel Iris Plus Graphics	650	macOS	1.37	96095.8080

```
[186]: df['Ram']=df['Ram'].astype('int32')
df['Weight']=df['Weight'].astype('float32')
```

```
[187]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1303 entries, 0 to 1302
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Company      1303 non-null   object
```

```

1  TypeName          1303 non-null  object
2  Inches            1303 non-null  float64
3  ScreenResolution  1303 non-null  object
4  Cpu               1303 non-null  object
5  Ram               1303 non-null  int32
6  Memory            1303 non-null  object
7  Gpu              1303 non-null  object
8  OpSys            1303 non-null  object
9  Weight           1303 non-null  float32
10 Price            1303 non-null  float64
dtypes: float32(1), float64(2), int32(1), object(7)
memory usage: 101.9+ KB

```

```
[188]: import seaborn as sns
import matplotlib.pyplot as plt
```

```
[189]: sns.distplot(df["Price"])
```

/tmp/ipykernel_75163/941010651.py:1: UserWarning:

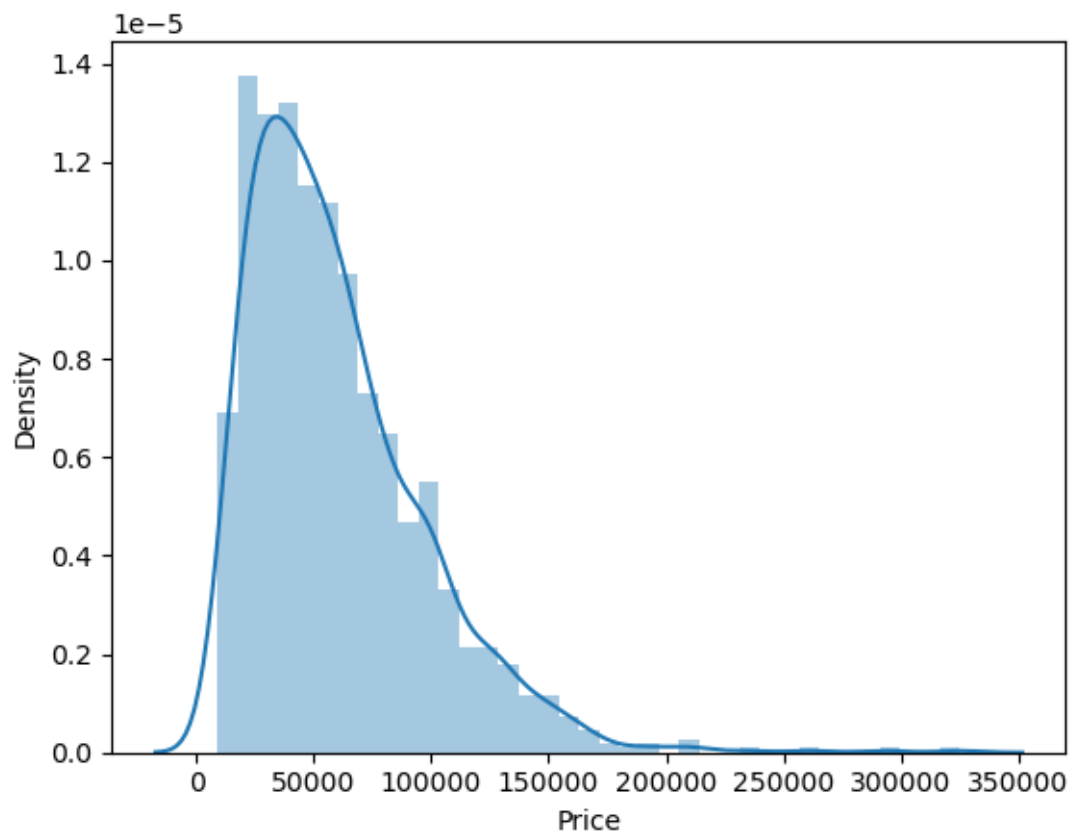
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

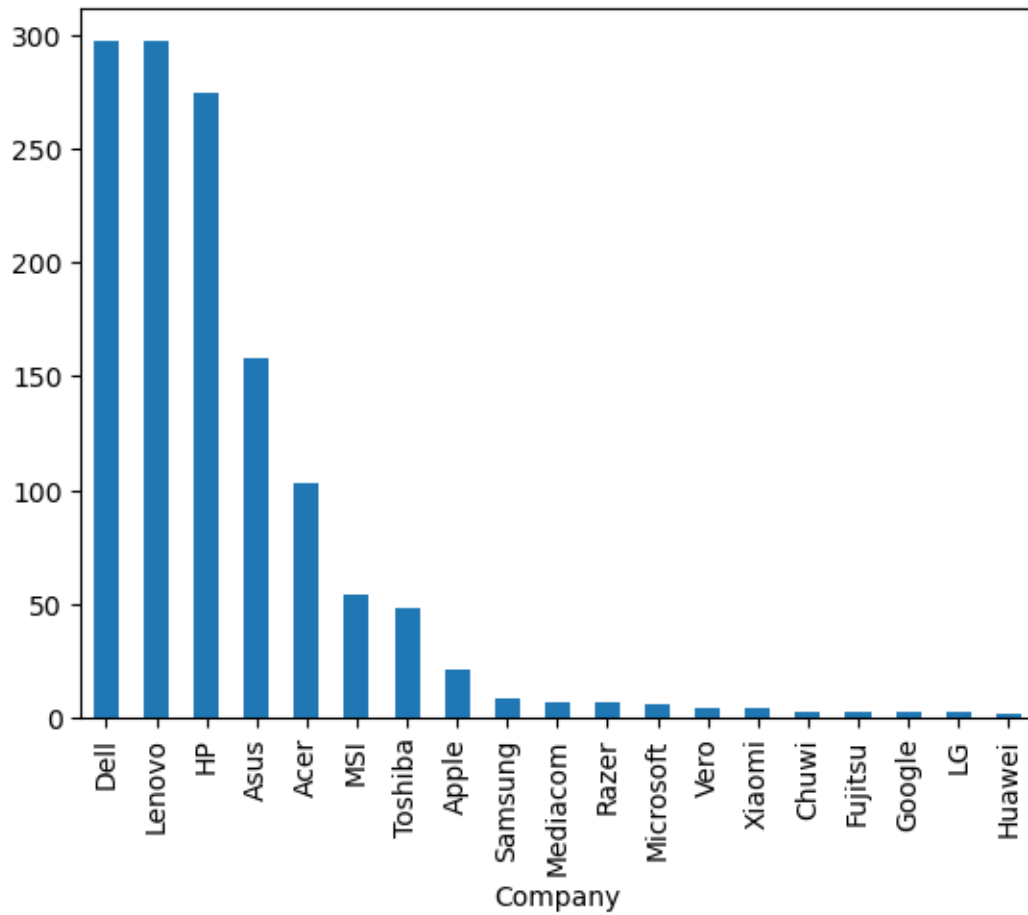
```
sns.distplot(df["Price"])
```

```
[189]: <AxesSubplot: xlabel='Price', ylabel='Density'>
```

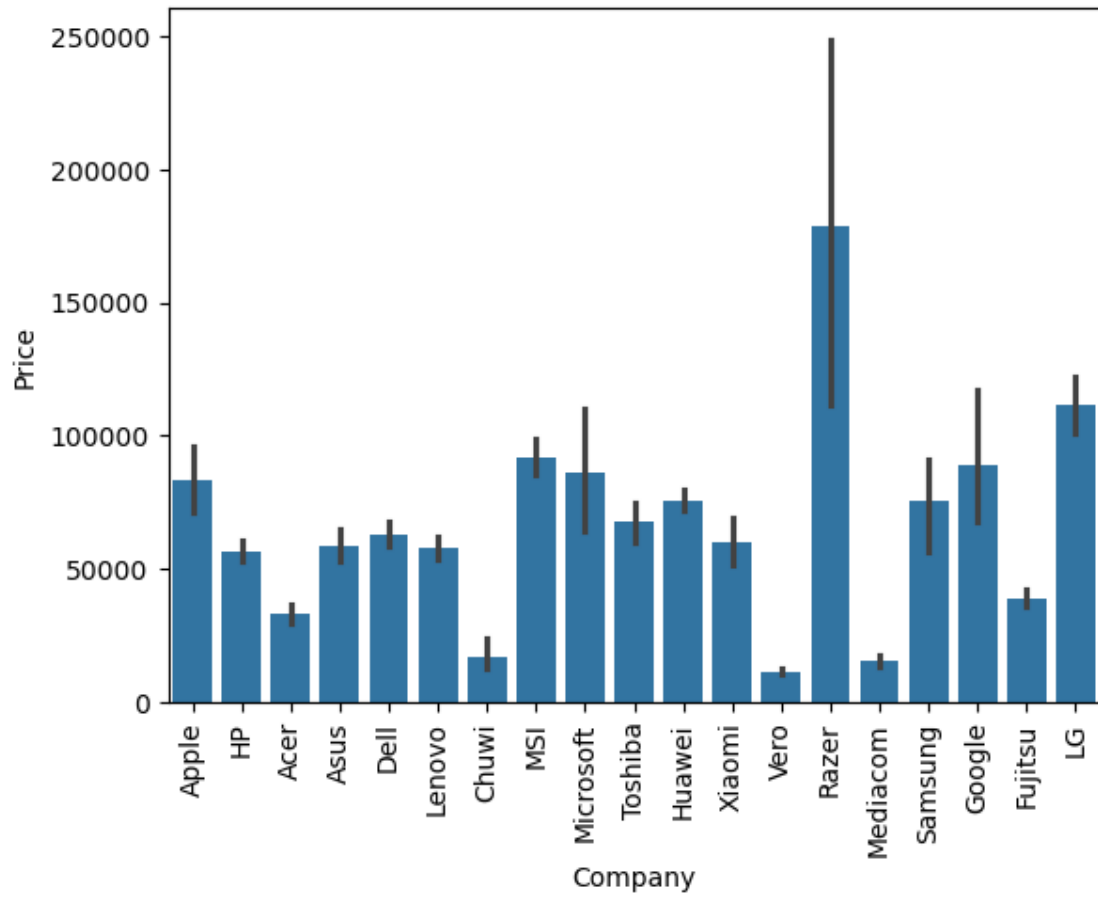


```
[190]: df['Company'].value_counts().plot(kind='bar')
```

```
[190]: <AxesSubplot: xlabel='Company'>
```

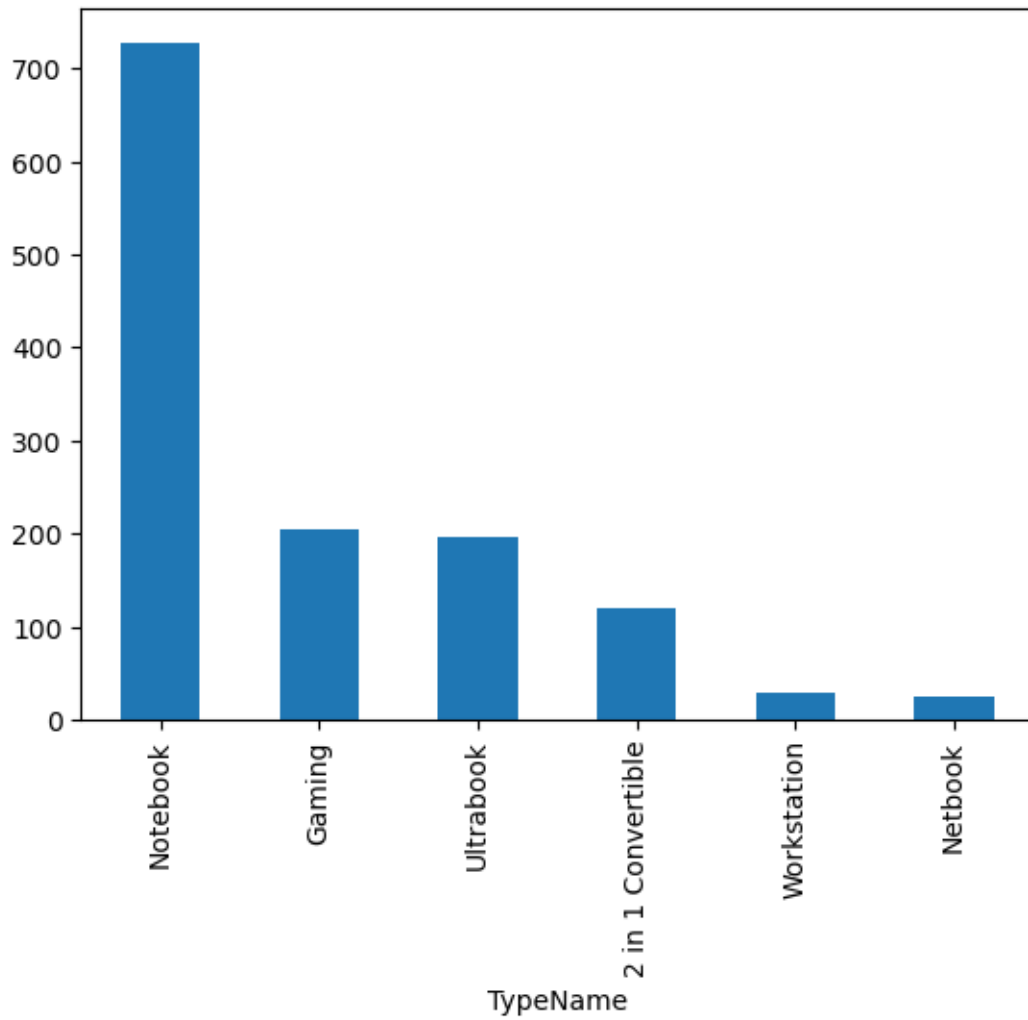


```
[191]: sns.barplot(x=df['Company'],y=df['Price'])  
plt.xticks(rotation='vertical')  
plt.show()
```

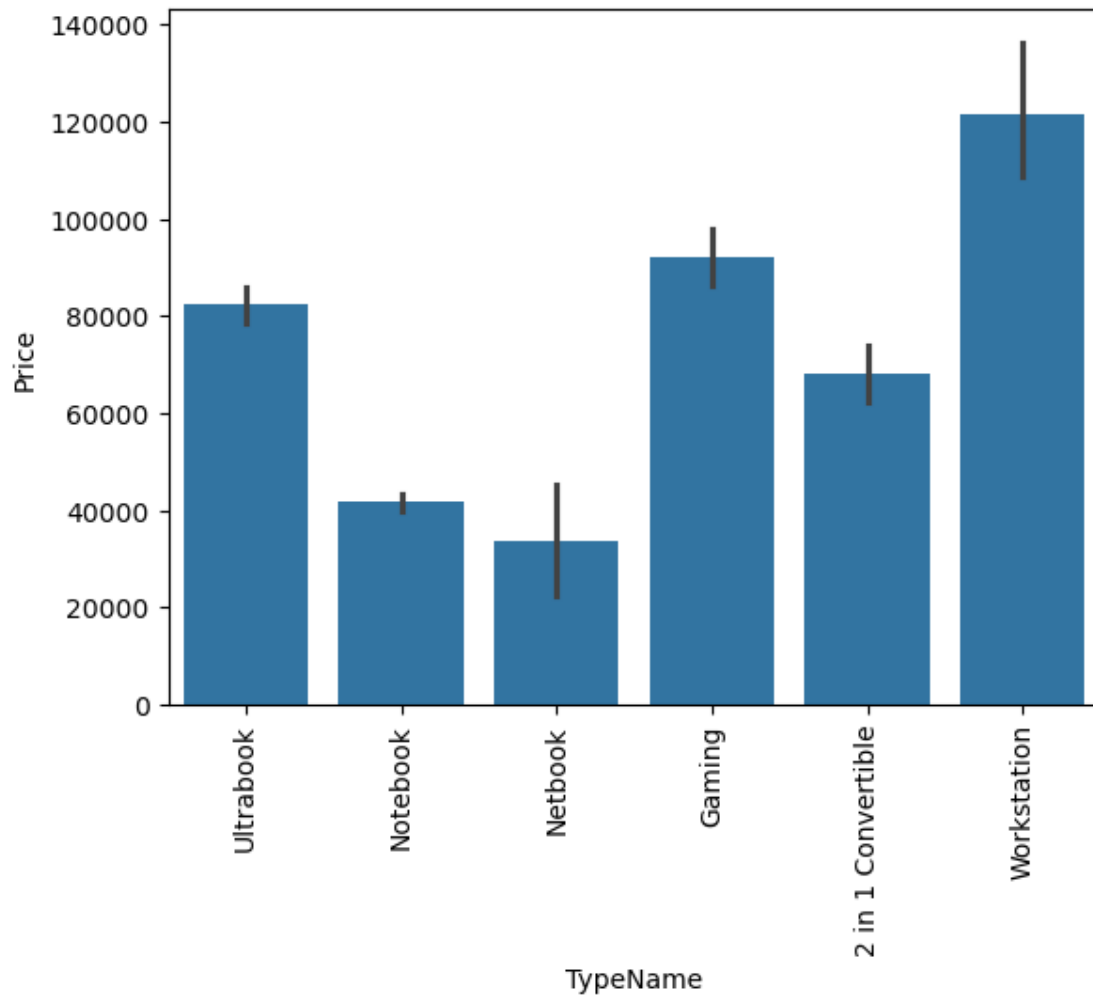


```
[192]: df['TypeName'].value_counts().plot(kind='bar')
```

```
[192]: <AxesSubplot: xlabel='TypeName'>
```



```
[193]: sns.barplot(x=df['TypeName'],y=df['Price'])  
plt.xticks(rotation='vertical')  
plt.show()
```

```
[194]: sns.distplot(df["Inches"])
```

/tmp/ipykernel_75163/480038204.py:1: UserWarning:

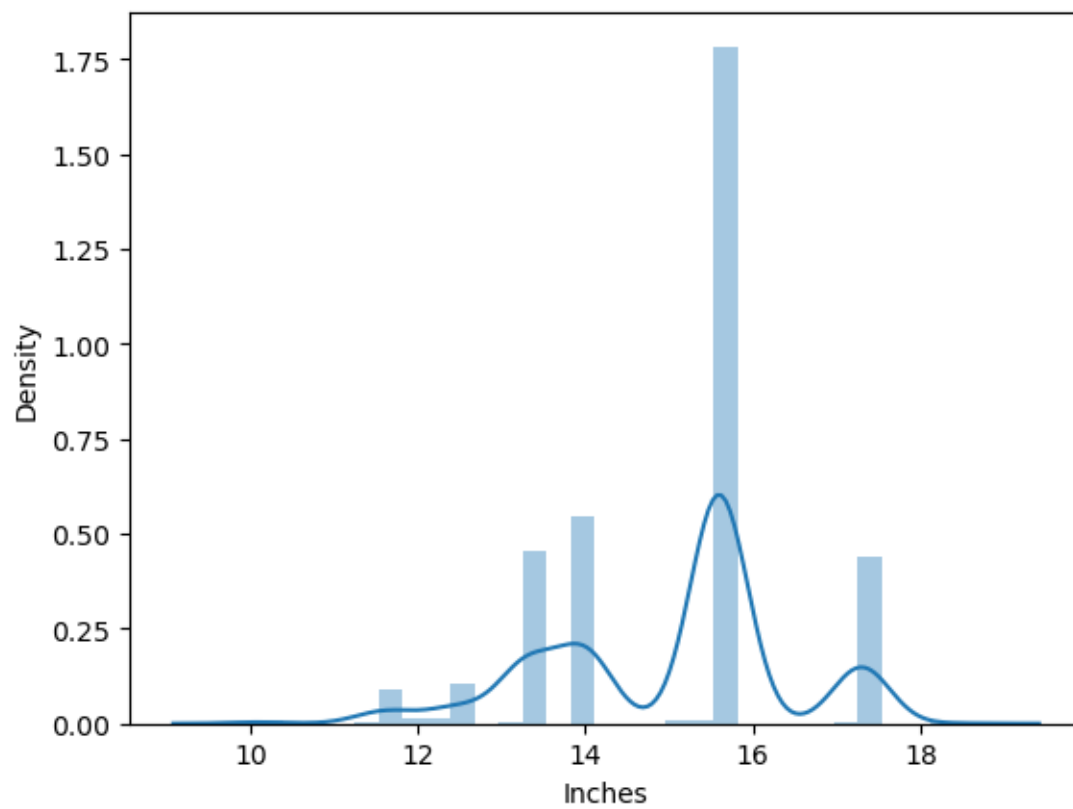
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

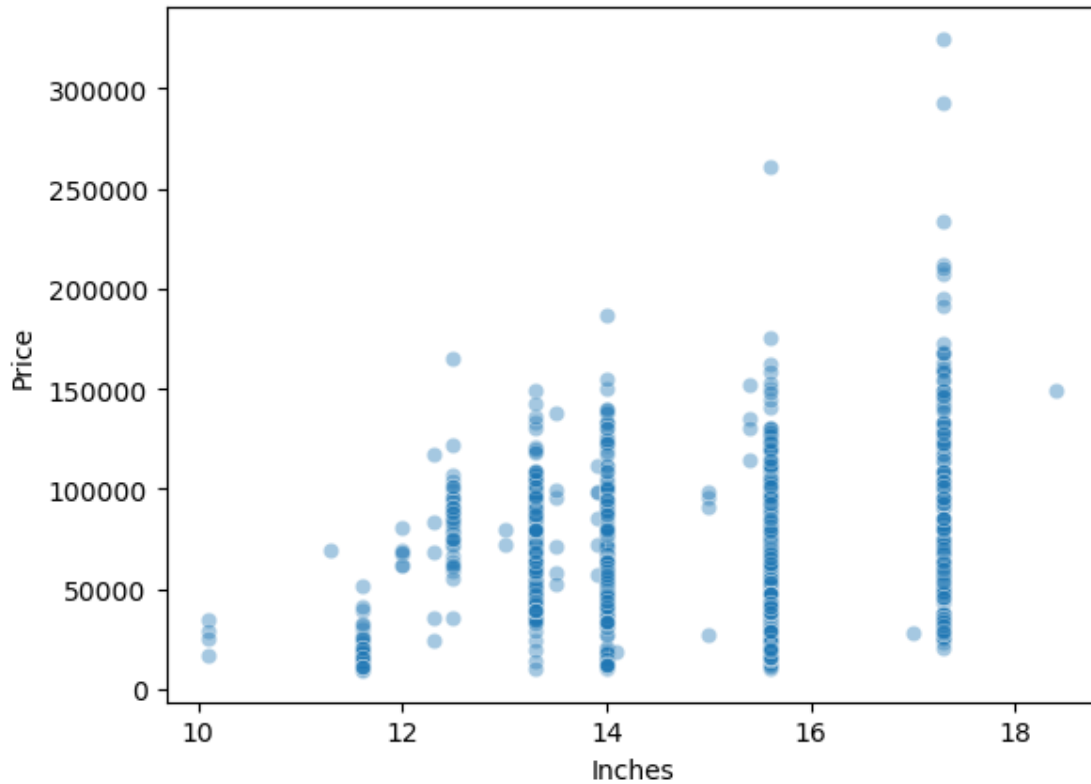
For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(df["Inches"])
```

```
[194]: <AxesSubplot: xlabel='Inches', ylabel='Density'>
```



```
[195]: sns.scatterplot(x=df['Inches'],y=df['Price'],alpha=0.4)  
plt.show()
```



```
[196]: df['ScreenResolution'].value_counts()
```

```
[196]: ScreenResolution
Full HD 1920x1080                    507
1366x768                             281
IPS Panel Full HD 1920x1080          230
IPS Panel Full HD / Touchscreen 1920x1080    53
Full HD / Touchscreen 1920x1080         47
1600x900                             23
Touchscreen 1366x768                 16
Quad HD+ / Touchscreen 3200x1800        15
IPS Panel 4K Ultra HD 3840x2160        12
IPS Panel 4K Ultra HD / Touchscreen 3840x2160  11
4K Ultra HD / Touchscreen 3840x2160      10
IPS Panel 1366x768                    7
Touchscreen 2560x1440                  7
4K Ultra HD 3840x2160                  7
IPS Panel Retina Display 2304x1440      6
IPS Panel Retina Display 2560x1600      6
Touchscreen 2256x1504                   6
IPS Panel Quad HD+ / Touchscreen 3200x1800  6
```

IPS Panel Touchscreen 2560x1440	5
IPS Panel Retina Display 2880x1800	4
1440x900	4
IPS Panel Touchscreen 1920x1200	4
IPS Panel 2560x1440	4
IPS Panel Quad HD+ 2560x1440	3
IPS Panel Touchscreen 1366x768	3
Quad HD+ 3200x1800	3
1920x1080	3
2560x1440	3
Touchscreen 2400x1600	3
IPS Panel Quad HD+ 3200x1800	2
IPS Panel Full HD 2160x1440	2
IPS Panel Touchscreen / 4K Ultra HD 3840x2160	2
IPS Panel Full HD 1366x768	1
Touchscreen / Quad HD+ 3200x1800	1
IPS Panel Retina Display 2736x1824	1
IPS Panel Full HD 2560x1440	1
IPS Panel Full HD 1920x1200	1
Touchscreen / Full HD 1920x1080	1
Touchscreen / 4K Ultra HD 3840x2160	1
IPS Panel Touchscreen 2400x1600	1

Name: count, dtype: int64

```
[197]: df['Touchscreen']=df['ScreenResolution'].apply(lambda x:1 if 'Touchscreen' in x
↳else 0)
```

```
[198]: df.sample(5)
```

```
[198]:
```

	Company	TypeName	Inches	ScreenResolution	\
428	HP	Gaming	17.3	Full HD 1920x1080	
231	HP	Notebook	15.6	1366x768	
813	Dell	Notebook	15.6	Full HD 1920x1080	
566	Dell	Notebook	15.6	1366x768	
1173	Lenovo	Notebook	15.6	1366x768	

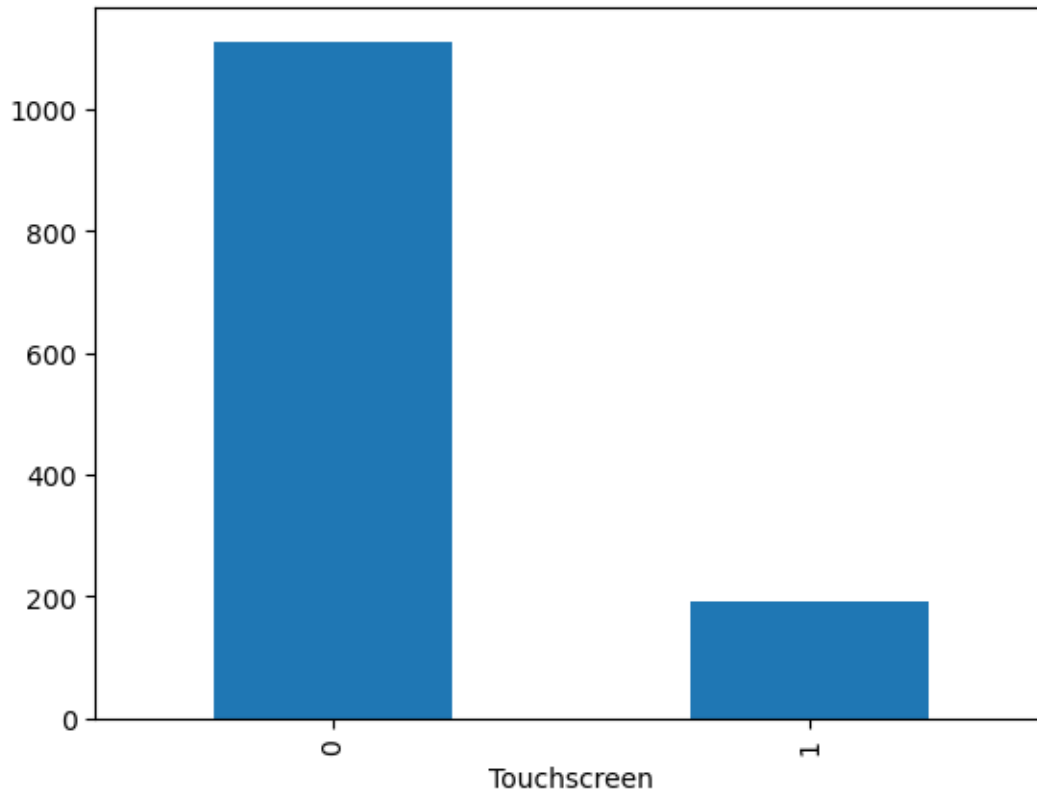
		Cpu	Ram	Memory	\
428	Intel Core i7 7700HQ	2.8GHz	12	256GB SSD + 1TB HDD	
231	AMD E-Series 9000e	1.5GHz	4	500GB HDD	
813	Intel Core i7 7500U	2.7GHz	8	1TB HDD	
566	Intel Core i5 7300U	2.6GHz	4	500GB HDD	
1173	Intel Core i5 6200U	2.3GHz	4	500GB HDD	

		Gpu	OpSys	Weight	Price	Touchscreen
428	Nvidia GeForce GTX 1070	Windows	10	3.35	106506.72	0
231	AMD Radeon R2	Windows	10	2.10	17582.40	0
813	Nvidia GeForce GT 940MX	Windows	10	1.98	51202.08	0

566	Intel HD Graphics 620	Windows 10	1.93	51095.52	0
1173	Intel HD Graphics 520	No OS	2.10	21205.44	0

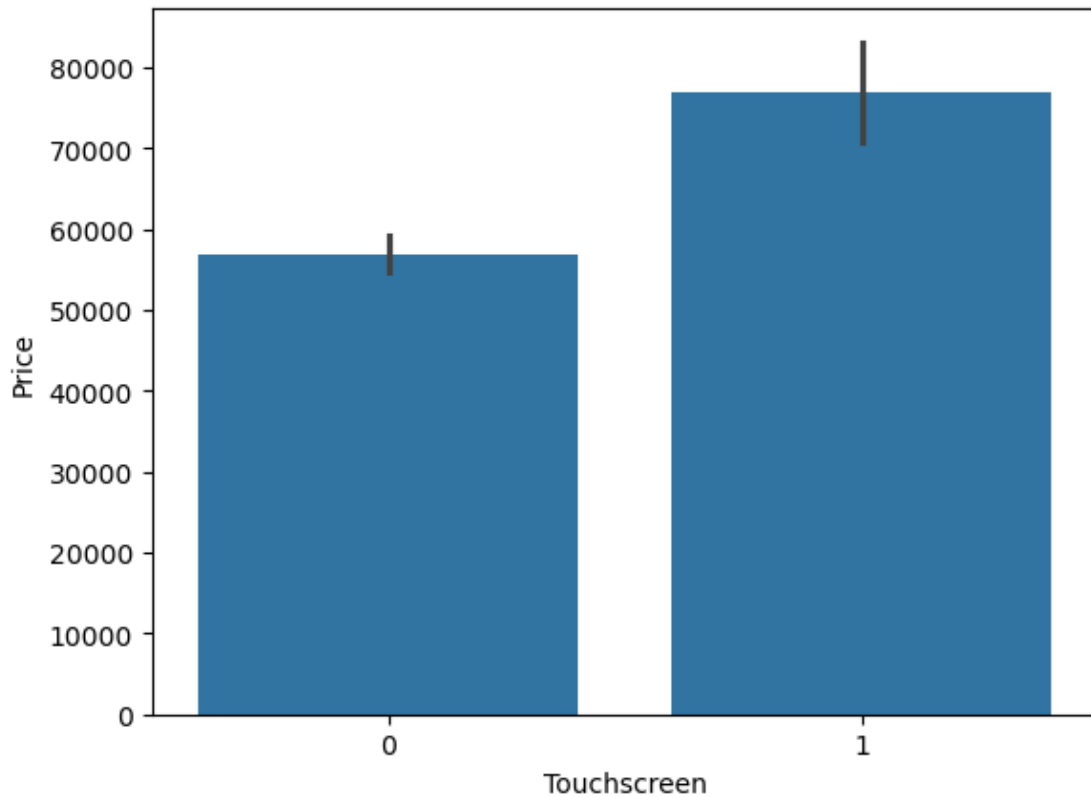
```
[199]: df['Touchscreen'].value_counts().plot(kind='bar')
```

```
[199]: <AxesSubplot: xlabel='Touchscreen'>
```



```
[200]: sns.barplot(x=df['Touchscreen'], y=df['Price'])
```

```
[200]: <AxesSubplot: xlabel='Touchscreen', ylabel='Price'>
```



```
[201]: df['Ips']=df['ScreenResolution'].apply(lambda x:1 if 'IPS' in x else 0)
```

```
[202]: df.head()
```

```
[202]:
```

	Company	TypeName	Inches	ScreenResolution
0	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600
1	Apple	Ultrabook	13.3	1440x900
2	HP	Notebook	15.6	Full HD 1920x1080
3	Apple	Ultrabook	15.4	IPS Panel Retina Display 2880x1800
4	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600

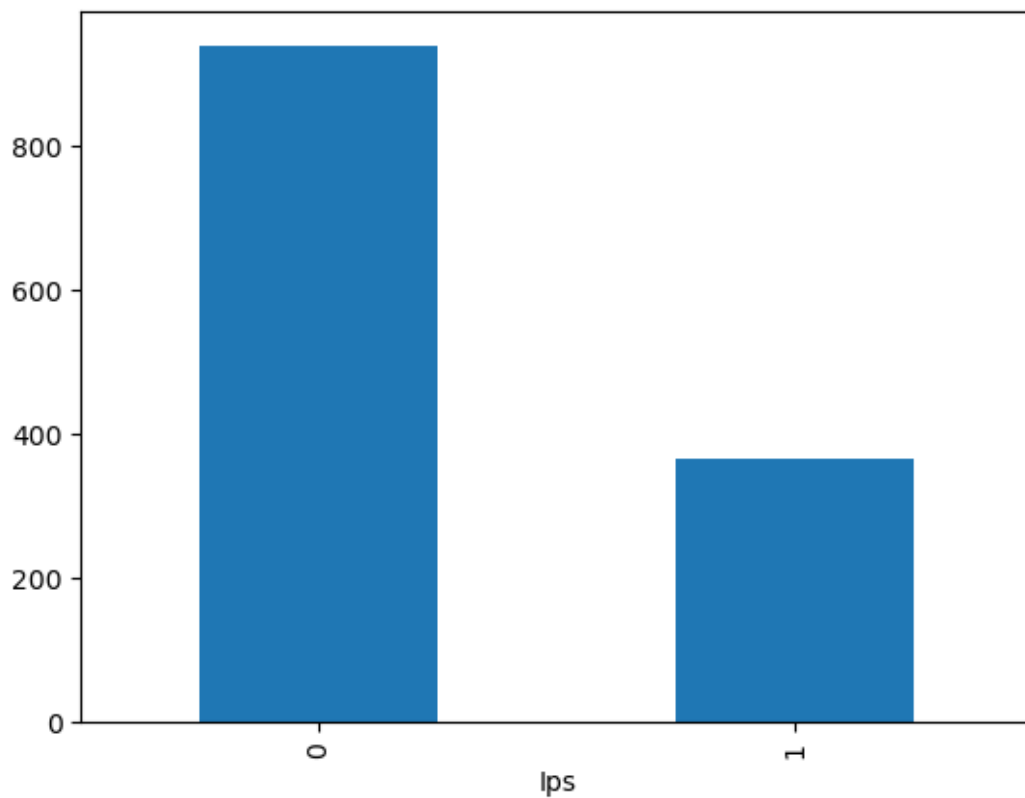
	Cpu	Ram	Memory
0	Intel Core i5 2.3GHz	8	128GB SSD
1	Intel Core i5 1.8GHz	8	128GB Flash Storage
2	Intel Core i5 7200U 2.5GHz	8	256GB SSD
3	Intel Core i7 2.7GHz	16	512GB SSD
4	Intel Core i5 3.1GHz	8	256GB SSD

	Gpu	OpSys	Weight	Price	Touchscreen	Ips
0	Intel Iris Plus Graphics 640	macOS	1.37	71378.6832	0	1
1	Intel HD Graphics 6000	macOS	1.34	47895.5232	0	0

2	Intel HD Graphics 620	No OS	1.86	30636.0000	0	0
3	AMD Radeon Pro 455	macOS	1.83	135195.3360	0	1
4	Intel Iris Plus Graphics 650	macOS	1.37	96095.8080	0	1

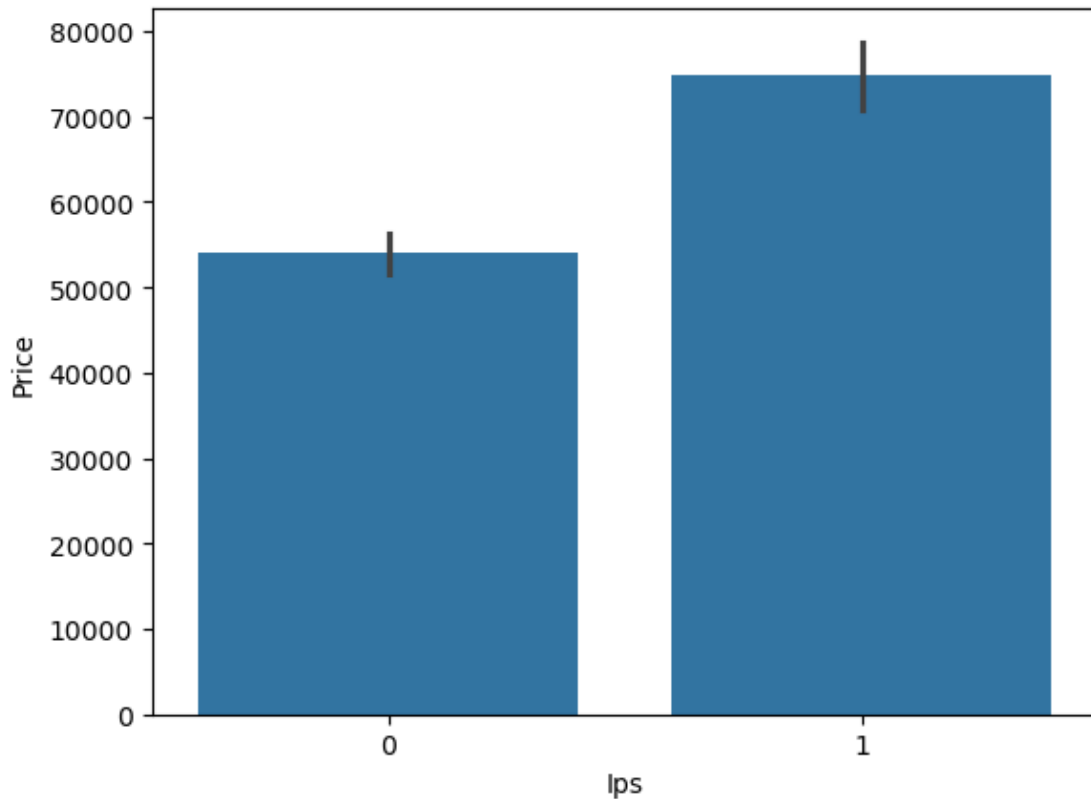
```
[203]: df['Ips'].value_counts().plot(kind='bar')
```

```
[203]: <AxesSubplot: xlabel='Ips'>
```



```
[204]: sns.barplot(x=df['Ips'],y=df['Price'])
```

```
[204]: <AxesSubplot: xlabel='Ips', ylabel='Price'>
```



```
[205]: new = df['ScreenResolution'].str.split('x',n=1,expand=True)
```

```
[206]: df['X_res'] = new[0]
df['Y_res'] = new[1]
```

```
[207]: df.head()
```

```
[207]: Company  TypeName  Inches  ScreenResolution \
0  Apple  Ultrabook   13.3  IPS Panel Retina Display 2560x1600
1  Apple  Ultrabook   13.3                    1440x900
2    HP   Notebook   15.6          Full HD 1920x1080
3  Apple  Ultrabook   15.4  IPS Panel Retina Display 2880x1800
4  Apple  Ultrabook   13.3  IPS Panel Retina Display 2560x1600

      Cpu  Ram  Memory \
0  Intel Core i5 2.3GHz  8  128GB SSD
1  Intel Core i5 1.8GHz  8  128GB Flash Storage
2  Intel Core i5 7200U 2.5GHz  8  256GB SSD
3  Intel Core i7 2.7GHz 16  512GB SSD
4  Intel Core i5 3.1GHz  8  256GB SSD
```


		Gpu	OpSys	Weight	Price	Touchscreen	Ips	\
0	Intel Iris Plus Graphics	640	macOS	1.37	71378.6832	0	1	
1	Intel HD Graphics	6000	macOS	1.34	47895.5232	0	0	
2	Intel HD Graphics	620	No OS	1.86	30636.0000	0	0	
3	AMD Radeon Pro	455	macOS	1.83	135195.3360	0	1	
4	Intel Iris Plus Graphics	650	macOS	1.37	96095.8080	0	1	

		X_res	Y_res
0	IPS Panel Retina Display	2560	1600
1		1440	900
2	Full HD	1920	1080
3	IPS Panel Retina Display	2880	1800
4	IPS Panel Retina Display	2560	1600

```
[208]: df['X_res'].str.replace(',','').str.findall(r'(\d+.\d+)').apply(lambda x:x[0])
```

```
[208]: 0      2560
      1      1440
      2      1920
      3      2880
      4      2560
      ...
     1298     1920
     1299     3200
     1300     1366
     1301     1366
     1302     1366
      Name: X_res, Length: 1303, dtype: object
```

```
[209]: df['X_res']=df['X_res'].str.replace(',','').str.findall(r'(\d+.\d+)').
      ↪ apply(lambda x:x[0])
```

```
[210]: df.head()
```

	Company	TypeName	Inches		ScreenResolution	\
0	Apple	Ultrabook	13.3	IPS Panel Retina Display	2560x1600	
1	Apple	Ultrabook	13.3		1440x900	
2	HP	Notebook	15.6	Full HD	1920x1080	
3	Apple	Ultrabook	15.4	IPS Panel Retina Display	2880x1800	
4	Apple	Ultrabook	13.3	IPS Panel Retina Display	2560x1600	

		Cpu	Ram	Memory	\
0	Intel Core i5	2.3GHz	8	128GB SSD	
1	Intel Core i5	1.8GHz	8	128GB Flash Storage	
2	Intel Core i5 7200U	2.5GHz	8	256GB SSD	
3	Intel Core i7	2.7GHz	16	512GB SSD	
4	Intel Core i5	3.1GHz	8	256GB SSD	

			Gpu	OpSys	Weight	Price	Touchscreen	Ips	\
0	Intel	Iris Plus Graphics	640	macOS	1.37	71378.6832	0	1	
1		Intel HD Graphics	6000	macOS	1.34	47895.5232	0	0	
2		Intel HD Graphics	620	No OS	1.86	30636.0000	0	0	
3		AMD Radeon Pro	455	macOS	1.83	135195.3360	0	1	
4	Intel	Iris Plus Graphics	650	macOS	1.37	96095.8080	0	1	

	X_res	Y_res
0	2560	1600
1	1440	900
2	1920	1080
3	2880	1800
4	2560	1600

```
[ ]:
```

```
[212]: df.head()
```

```
[212]: Company      TypeName  Inches      ScreenResolution \
0   Apple  Ultrabook    13.3  IPS Panel Retina Display 2560x1600
1   Apple  Ultrabook    13.3                1440x900
2    HP    Notebook    15.6                Full HD 1920x1080
3   Apple  Ultrabook    15.4  IPS Panel Retina Display 2880x1800
4   Apple  Ultrabook    13.3  IPS Panel Retina Display 2560x1600
```

		Cpu	Ram	Memory	\
0	Intel Core i5	2.3GHz	8	128GB SSD	
1	Intel Core i5	1.8GHz	8	128GB Flash Storage	
2	Intel Core i5 7200U	2.5GHz	8	256GB SSD	
3	Intel Core i7	2.7GHz	16	512GB SSD	
4	Intel Core i5	3.1GHz	8	256GB SSD	

			Gpu	OpSys	Weight	Price	Touchscreen	Ips	\
0	Intel	Iris Plus Graphics	640	macOS	1.37	71378.6832	0	1	
1		Intel HD Graphics	6000	macOS	1.34	47895.5232	0	0	
2		Intel HD Graphics	620	No OS	1.86	30636.0000	0	0	
3		AMD Radeon Pro	455	macOS	1.83	135195.3360	0	1	
4	Intel	Iris Plus Graphics	650	macOS	1.37	96095.8080	0	1	

	X_res	Y_res
0	2560	1600
1	1440	900
2	1920	1080
3	2880	1800
4	2560	1600

```
[213]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1303 entries, 0 to 1302
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Company               1303 non-null  object
1   TypeName              1303 non-null  object
2   Inches                1303 non-null  float64
3   ScreenResolution      1303 non-null  object
4   Cpu                   1303 non-null  object
5   Ram                   1303 non-null  int32
6   Memory                1303 non-null  object
7   Gpu                   1303 non-null  object
8   OpSys                 1303 non-null  object
9   Weight                1303 non-null  float32
10  Price                 1303 non-null  float64
11  Touchscreen           1303 non-null  int64
12  Ips                   1303 non-null  int64
13  X_res                 1303 non-null  object
14  Y_res                 1303 non-null  object
dtypes: float32(1), float64(2), int32(1), int64(2), object(9)
memory usage: 142.6+ KB
```

```
[214]: df['X_res']=df['X_res'].astype('int32')
df['Y_res']=df['Y_res'].astype('int32')
```

```
[215]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1303 entries, 0 to 1302
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Company               1303 non-null  object
1   TypeName              1303 non-null  object
2   Inches                1303 non-null  float64
3   ScreenResolution      1303 non-null  object
4   Cpu                   1303 non-null  object
5   Ram                   1303 non-null  int32
6   Memory                1303 non-null  object
7   Gpu                   1303 non-null  object
8   OpSys                 1303 non-null  object
9   Weight                1303 non-null  float32
10  Price                 1303 non-null  float64
11  Touchscreen           1303 non-null  int64
12  Ips                   1303 non-null  int64
```

```

13  X_res          1303 non-null   int32
14  Y_res          1303 non-null   int32
dtypes: float32(1), float64(2), int32(3), int64(2), object(7)
memory usage: 132.5+ KB

```

```
[216]: df.select_dtypes(include='number').corr()['Price']
```

```

[216]: Inches          0.068197
      Ram             0.743007
      Weight          0.210370
      Price           1.000000
      Touchscreen     0.191226
      Ips             0.252208
      X_res           0.556529
      Y_res           0.552809
      Name: Price, dtype: float64

```

```

[217]: df['ppi']=np.sqrt((df['X_res']**2)+(df['Y_res']**2))/df['Inches'].
      ↪astype('float')

```

```
[218]: df.head()
```

```

[218]: Company  TypeName  Inches          ScreenResolution \
0   Apple  Ultrabook   13.3  IPS Panel Retina Display 2560x1600
1   Apple  Ultrabook   13.3                    1440x900
2    HP    Notebook   15.6          Full HD 1920x1080
3   Apple  Ultrabook   15.4  IPS Panel Retina Display 2880x1800
4   Apple  Ultrabook   13.3  IPS Panel Retina Display 2560x1600

          Cpu  Ram          Memory \
0      Intel Core i5 2.3GHz      8      128GB SSD
1      Intel Core i5 1.8GHz      8  128GB Flash Storage
2  Intel Core i5 7200U 2.5GHz      8      256GB SSD
3      Intel Core i7 2.7GHz     16      512GB SSD
4      Intel Core i5 3.1GHz      8      256GB SSD

          Gpu  OpSys  Weight      Price  Touchscreen  Ips \
0  Intel Iris Plus Graphics 640  macOS    1.37  71378.6832      0    1
1      Intel HD Graphics 6000  macOS    1.34  47895.5232      0    0
2      Intel HD Graphics 620   No OS    1.86  30636.0000      0    0
3      AMD Radeon Pro 455   macOS    1.83  135195.3360      0    1
4  Intel Iris Plus Graphics 650  macOS    1.37  96095.8080      0    1

      X_res  Y_res      ppi
0    2560   1600  226.983005
1    1440    900  127.677940
2    1920  1080  141.211998
3    2880  1800  220.534624

```

```
4    2560    1600    226.983005
```

```
[219]: df.select_dtypes(include='number').corr()['Price']
```

```
[219]: Inches          0.068197
Ram              0.743007
Weight          0.210370
Price           1.000000
Touchscreen     0.191226
Ips             0.252208
X_res           0.556529
Y_res           0.552809
ppi            0.473487
Name: Price, dtype: float64
```

```
[220]: df.drop(columns=["ScreenResolution"],inplace=True)
```

```
[221]: df.head()
```

```
[221]: Company  TypeName  Inches          Cpu  Ram  \
0   Apple  Ultrabook   13.3      Intel Core i5 2.3GHz    8
1   Apple  Ultrabook   13.3      Intel Core i5 1.8GHz    8
2    HP    Notebook   15.6  Intel Core i5 7200U 2.5GHz    8
3   Apple  Ultrabook   15.4      Intel Core i7 2.7GHz   16
4   Apple  Ultrabook   13.3      Intel Core i5 3.1GHz    8

      Memory          Gpu  OpSys  Weight  \
0      128GB SSD  Intel Iris Plus Graphics 640  macOS    1.37
1  128GB Flash Storage    Intel HD Graphics 6000  macOS    1.34
2      256GB SSD    Intel HD Graphics 620  No OS    1.86
3      512GB SSD      AMD Radeon Pro 455  macOS    1.83
4      256GB SSD  Intel Iris Plus Graphics 650  macOS    1.37

      Price  Touchscreen  Ips  X_res  Y_res      ppi
0   71378.6832         0    1   2560   1600  226.983005
1   47895.5232         0    0   1440    900  127.677940
2   30636.0000         0    0   1920   1080  141.211998
3  135195.3360         0    1   2880   1800  220.534624
4   96095.8080         0    1   2560   1600  226.983005
```

```
[222]: df.drop(columns=["Inches", "X_res", "Y_res"],inplace=True)
```

```
[223]: df.head()
```

```
[223]: Company  TypeName          Cpu  Ram      Memory  \
0   Apple  Ultrabook      Intel Core i5 2.3GHz    8      128GB SSD
1   Apple  Ultrabook      Intel Core i5 1.8GHz    8  128GB Flash Storage
2    HP    Notebook  Intel Core i5 7200U 2.5GHz    8      256GB SSD
```

3	Apple	Ultrabook	Intel Core i7 2.7GHz	16	512GB SSD
4	Apple	Ultrabook	Intel Core i5 3.1GHz	8	256GB SSD

		Gpu	OpSys	Weight	Price	Touchscreen	Ips	\
0	Intel	Iris Plus Graphics 640	macOS	1.37	71378.6832	0	1	
1		Intel HD Graphics 6000	macOS	1.34	47895.5232	0	0	
2		Intel HD Graphics 620	No OS	1.86	30636.0000	0	0	
3		AMD Radeon Pro 455	macOS	1.83	135195.3360	0	1	
4	Intel	Iris Plus Graphics 650	macOS	1.37	96095.8080	0	1	

ppi

0	226.983005
1	127.677940
2	141.211998
3	220.534624
4	226.983005

```
[224]: df['Cpu'].value_counts()
```

```
[224]: Cpu
Intel Core i5 7200U 2.5GHz      190
Intel Core i7 7700HQ 2.8GHz    146
Intel Core i7 7500U 2.7GHz     134
Intel Core i7 8550U 1.8GHz      73
Intel Core i5 8250U 1.6GHz      72
...
Intel Core M M7-6Y75 1.2GHz      1
Intel Core i5 7200U 2.50GHz      1
Intel Core i5 7200U 2.70GHz      1
Intel Core i5 7200U 2.7GHz      1
Intel Core i7 2.7GHz            1
Name: count, Length: 118, dtype: int64
```

```
[225]: df['Cpu Name']=df['Cpu'].apply(lambda x:" ".join(x.split()[0:3]))
```

```
[226]: df.head()
```

```
[226]: Company  TypeName                Cpu  Ram      Memory \
0  Apple  Ultrabook      Intel Core i5 2.3GHz    8      128GB SSD
1  Apple  Ultrabook      Intel Core i5 1.8GHz    8  128GB Flash Storage
2   HP    Notebook  Intel Core i5 7200U 2.5GHz    8      256GB SSD
3  Apple  Ultrabook      Intel Core i7 2.7GHz   16      512GB SSD
4  Apple  Ultrabook      Intel Core i5 3.1GHz    8      256GB SSD
```

		Gpu	OpSys	Weight	Price	Touchscreen	Ips	\
0	Intel	Iris Plus Graphics 640	macOS	1.37	71378.6832	0	1	
1		Intel HD Graphics 6000	macOS	1.34	47895.5232	0	0	

2	Intel HD Graphics 620	No OS	1.86	30636.0000	0	0
3	AMD Radeon Pro 455	macOS	1.83	135195.3360	0	1
4	Intel Iris Plus Graphics 650	macOS	1.37	96095.8080	0	1

	ppi	Cpu Name
0	226.983005	Intel Core i5
1	127.677940	Intel Core i5
2	141.211998	Intel Core i5
3	220.534624	Intel Core i7
4	226.983005	Intel Core i5

```
[227]: def fetch_processor(text):
        if text == 'Intel Core i7' or text == 'Intel Core i5' or text == "Intel_
        Core i3":
            return text
        else:
            if text.split()[0] == 'Intel':
                return 'Other Intel Processor'
            else:
                return 'Amd Processor'
```

```
[228]: df['Cpu brand'] = df['Cpu Name'].apply(fetch_processor)
```

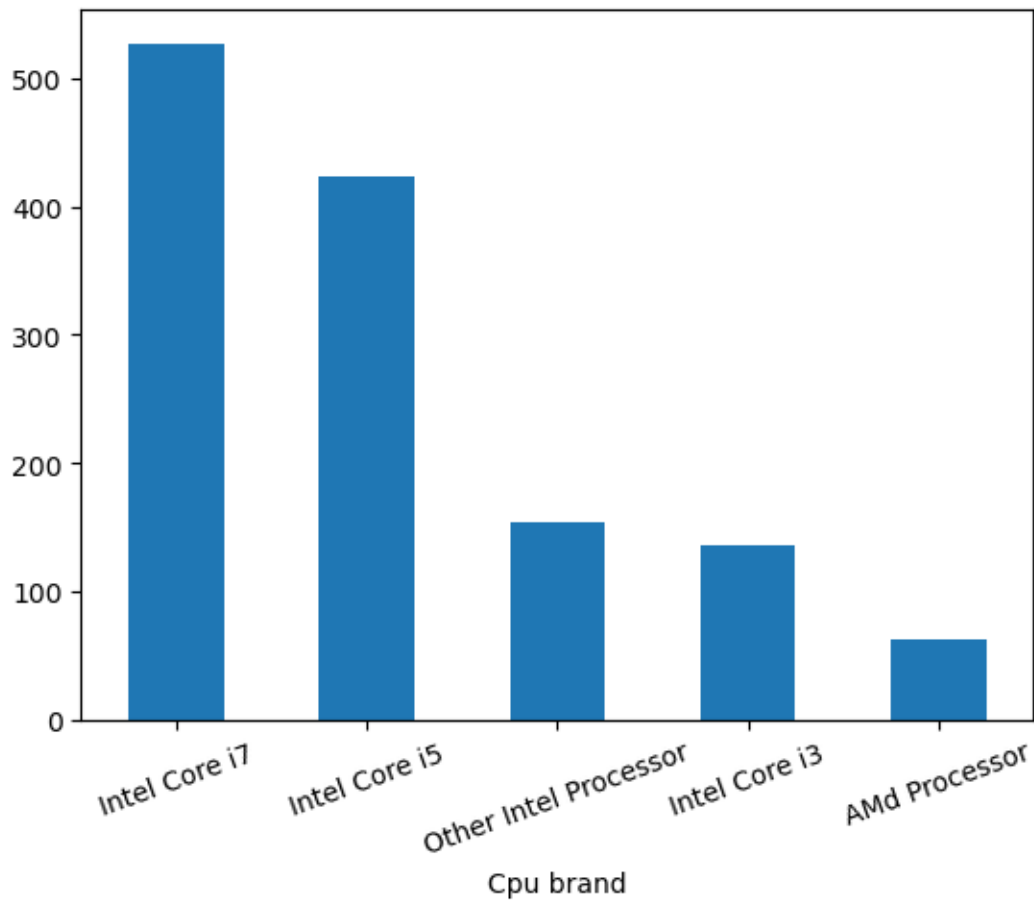
```
[229]: df.head()
```

```
[229]: Company  TypeName          Cpu  Ram          Memory \
0  Apple  Ultrabook      Intel Core i5 2.3GHz    8          128GB SSD
1  Apple  Ultrabook      Intel Core i5 1.8GHz    8  128GB Flash Storage
2   HP    Notebook  Intel Core i5 7200U 2.5GHz    8          256GB SSD
3  Apple  Ultrabook      Intel Core i7 2.7GHz   16          512GB SSD
4  Apple  Ultrabook      Intel Core i5 3.1GHz    8          256GB SSD

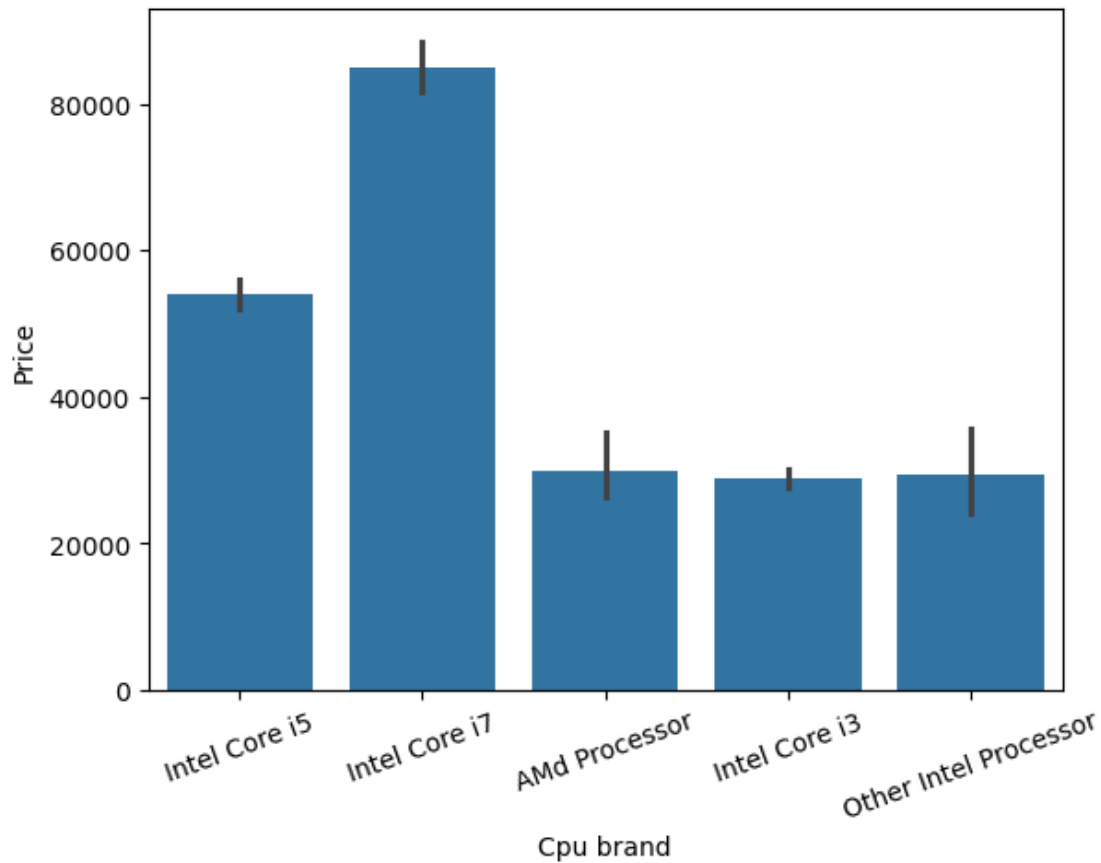
          Gpu  OpSys  Weight          Price  Touchscreen  Ips \
0  Intel Iris Plus Graphics 640  macOS    1.37   71378.6832         0    1
1      Intel HD Graphics 6000  macOS    1.34   47895.5232         0    0
2      Intel HD Graphics 620  No OS    1.86   30636.0000         0    0
3      AMD Radeon Pro 455  macOS    1.83  135195.3360         0    1
4  Intel Iris Plus Graphics 650  macOS    1.37   96095.8080         0    1

          ppi          Cpu Name      Cpu brand
0  226.983005  Intel Core i5  Intel Core i5
1  127.677940  Intel Core i5  Intel Core i5
2  141.211998  Intel Core i5  Intel Core i5
3  220.534624  Intel Core i7  Intel Core i7
4  226.983005  Intel Core i5  Intel Core i5
```

```
[230]: df['Cpu brand'].value_counts().plot(kind='bar')  
plt.xticks(rotation=20)  
plt.show()
```



```
[231]: sns.barplot(x=df['Cpu brand'],y=df['Price'])  
plt.xticks(rotation=20)  
plt.show()
```

```
[232]: df.drop(columns=['Cpu', 'Cpu Name'], inplace=True)
```

```
[233]: df.head()
```

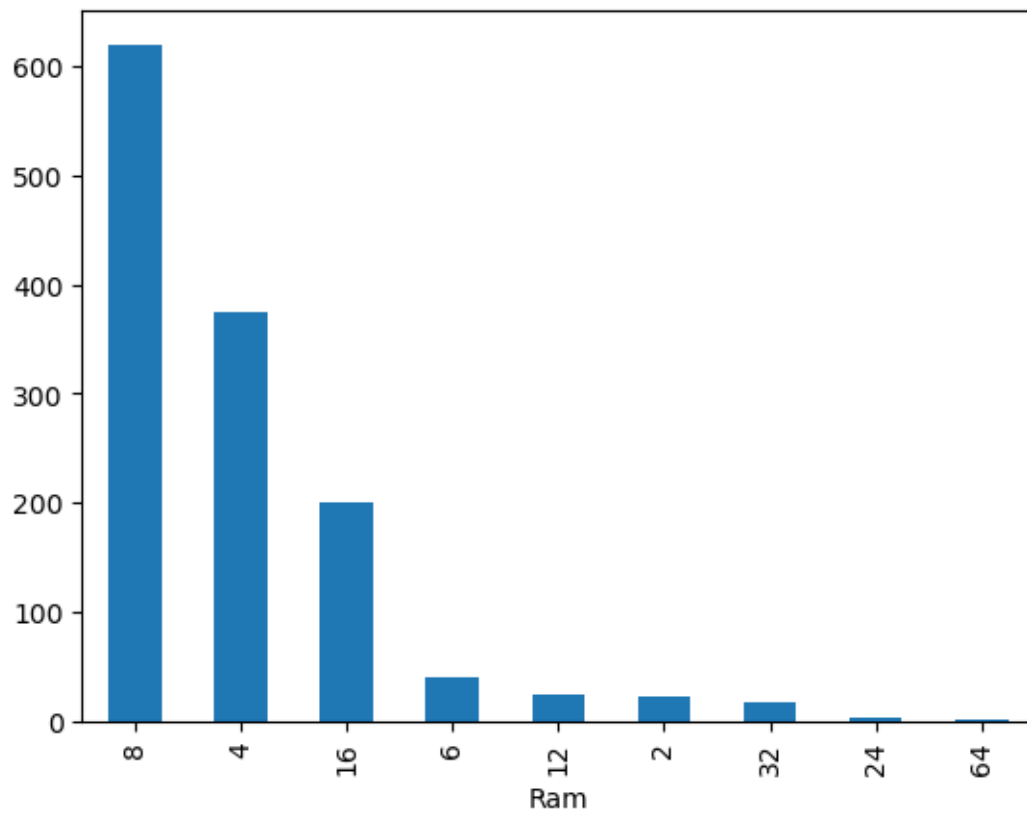
```
[233]:
```

	Company	TypeName	Ram	Memory	Gpu \
0	Apple	Ultrabook	8	128GB SSD	Intel Iris Plus Graphics 640
1	Apple	Ultrabook	8	128GB Flash Storage	Intel HD Graphics 6000
2	HP	Notebook	8	256GB SSD	Intel HD Graphics 620
3	Apple	Ultrabook	16	512GB SSD	AMD Radeon Pro 455
4	Apple	Ultrabook	8	256GB SSD	Intel Iris Plus Graphics 650

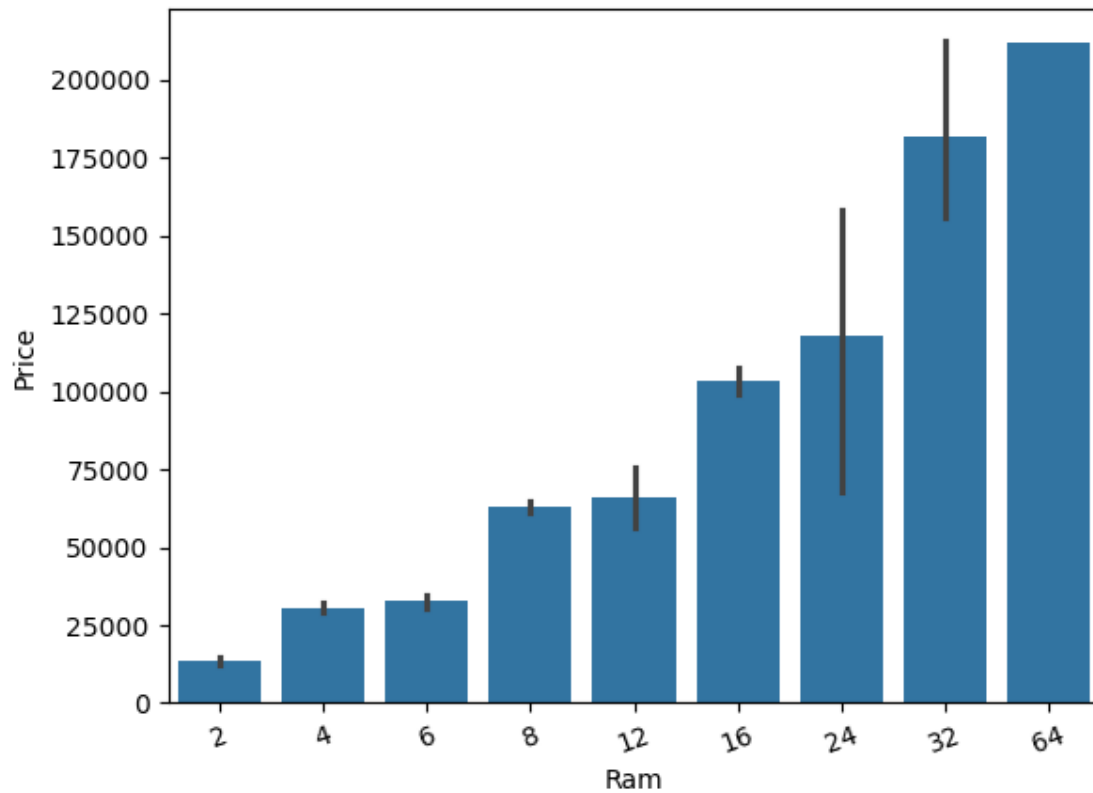
	OpSys	Weight	Price	Touchscreen	Ips	ppi	Cpu brand
0	macOS	1.37	71378.6832	0	1	226.983005	Intel Core i5
1	macOS	1.34	47895.5232	0	0	127.677940	Intel Core i5
2	No OS	1.86	30636.0000	0	0	141.211998	Intel Core i5
3	macOS	1.83	135195.3360	0	1	220.534624	Intel Core i7
4	macOS	1.37	96095.8080	0	1	226.983005	Intel Core i5

```
[234]: df['Ram'].value_counts().plot(kind='bar')
```

[234]: <AxesSubplot: xlabel='Ram'>



```
[235]: sns.barplot(x=df['Ram'],y=df['Price'])  
plt.xticks(rotation=20)  
plt.show()
```



```
[236]: df['Memory'].value_counts()
```

```
[236]: Memory
256GB SSD          412
1TB HDD            223
500GB HDD          132
512GB SSD          118
128GB SSD + 1TB HDD  94
128GB SSD           76
256GB SSD + 1TB HDD  73
32GB Flash Storage  38
2TB HDD             16
64GB Flash Storage  15
1TB SSD             14
512GB SSD + 1TB HDD  14
256GB SSD + 2TB HDD  10
1.0TB Hybrid        9
256GB Flash Storage  8
16GB Flash Storage  7
32GB SSD             6
180GB SSD            5
```

128GB Flash Storage	4
16GB SSD	3
512GB SSD + 2TB HDD	3
128GB SSD + 2TB HDD	2
256GB SSD + 256GB SSD	2
512GB Flash Storage	2
1TB SSD + 1TB HDD	2
256GB SSD + 500GB HDD	2
64GB SSD	1
512GB SSD + 512GB SSD	1
64GB Flash Storage + 1TB HDD	1
1TB HDD + 1TB HDD	1
512GB SSD + 256GB SSD	1
32GB HDD	1
128GB HDD	1
240GB SSD	1
8GB SSD	1
508GB Hybrid	1
1.0TB HDD	1
512GB SSD + 1.0TB Hybrid	1
256GB SSD + 1.0TB Hybrid	1

Name: count, dtype: int64

```
[237]: df['Memory'] = df['Memory'].astype(str).replace('\.0', '', regex=True)
df["Memory"] = df["Memory"].str.replace('GB', '')
df["Memory"] = df["Memory"].str.replace('TB', '000')

new = df["Memory"].str.split("+", n=1, expand=True)
df["first"] = new[0].str.strip()
df["second"] = new[1]

df["Layer1HDD"] = df["first"].apply(lambda x: 1 if "HDD" in x else 0)
df["Layer1SSD"] = df["first"].apply(lambda x: 1 if "SSD" in x else 0)
df["Layer1Hybrid"] = df["first"].apply(lambda x: 1 if "Hybrid" in x else 0)
df["Layer1Flash_Storage"] = df["first"].apply(lambda x: 1 if "Flash Storage" in
↪x else 0)

df['first'] = df['first'].str.extract('(\d+)')
df['first'] = df['first'].fillna(0).astype(int)

df["second"] = df["second"].fillna("0")

df["Layer2HDD"] = df["second"].apply(lambda x: 1 if isinstance(x, str) and
↪ "HDD" in x else 0)
df["Layer2SSD"] = df["second"].apply(lambda x: 1 if isinstance(x, str) and
↪ "SSD" in x else 0)
```

```

df["Layer2Hybrid"] = df["second"].apply(lambda x: 1 if isinstance(x, str) and
    ↪ "Hybrid" in x else 0)
df["Layer2Flash_Storage"] = df["second"].apply(lambda x: 1 if isinstance(x,
    ↪ str) and "Flash Storage" in x else 0)

df['second'] = df['second'].str.extract('(\d+)') # Extract only numeric values
df['second'] = df['second'].fillna(0).astype(int) # Convert to integer safely

df["HDD"] = (df["first"] * df["Layer1HDD"] + df["second"] * df["Layer2HDD"])
df["SSD"] = (df["first"] * df["Layer1SSD"] + df["second"] * df["Layer2SSD"])
df["Hybrid"] = (df["first"] * df["Layer1Hybrid"] + df["second"] *
    ↪ df["Layer2Hybrid"])
df["Flash_Storage"] = (df["first"] * df["Layer1Flash_Storage"] + df["second"] *
    ↪ df["Layer2Flash_Storage"])

df.drop(columns=['first', 'second', 'Layer1HDD', 'Layer1SSD', 'Layer1Hybrid',
    ↪ 'Layer1Flash_Storage', 'Layer2HDD', 'Layer2SSD',
    ↪ 'Layer2Hybrid',
    ↪ 'Layer2Flash_Storage'], inplace=True)

```

```

<>:1: SyntaxWarning: invalid escape sequence '\.'
<>:17: SyntaxWarning: invalid escape sequence '\d'
<>:28: SyntaxWarning: invalid escape sequence '\d'
<>:1: SyntaxWarning: invalid escape sequence '\.'
<>:17: SyntaxWarning: invalid escape sequence '\d'
<>:28: SyntaxWarning: invalid escape sequence '\d'
/tmp/ipykernel_75163/1007893785.py:1: SyntaxWarning: invalid escape sequence
'\.'
    df['Memory'] = df['Memory'].astype(str).replace('\.0', '', regex=True)
/tmp/ipykernel_75163/1007893785.py:17: SyntaxWarning: invalid escape sequence
'\d'
    df['first'] = df['first'].str.extract('(\d+)') # Extract only numbers
/tmp/ipykernel_75163/1007893785.py:28: SyntaxWarning: invalid escape sequence
'\d'
    df['second'] = df['second'].str.extract('(\d+)') # Extract only numeric
values

```

[241]: df.sample()

```

[241]:      Company TypeName  Ram      Memory      Gpu \
153    MSI    Gaming   16  256 SSD + 1000 HDD  Nvidia GeForce GTX 1060

      OpSys  Weight  Price  Touchscreen  Ips      ppi \
153  Windows 10    2.8  100699.2         0    0  127.335675

      Cpu brand  HDD  SSD  Hybrid  Flash_Storage
153  Intel Core i7  1000  256      0              0

```

```
[242]: df.drop(columns=['Memory'],inplace=True)
```

```
[243]: df.head()
```

```
[243]: Company   TypeName  Ram      Gpu  OpSys  Weight  \
0   Apple  Ultrabook    8  Intel Iris Plus Graphics 640  macOS    1.37
1   Apple  Ultrabook    8      Intel HD Graphics 6000  macOS    1.34
2    HP    Notebook    8      Intel HD Graphics 620  No OS    1.86
3   Apple  Ultrabook   16      AMD Radeon Pro 455  macOS    1.83
4   Apple  Ultrabook    8  Intel Iris Plus Graphics 650  macOS    1.37

      Price  Touchscreen  Ips      ppi      Cpu brand  HDD  SSD  Hybrid  \
0  71378.6832          0    1  226.983005  Intel Core i5    0  128      0
1  47895.5232          0    0  127.677940  Intel Core i5    0    0      0
2  30636.0000          0    0  141.211998  Intel Core i5    0  256      0
3  135195.3360          0    1  220.534624  Intel Core i7    0  512      0
4   96095.8080          0    1  226.983005  Intel Core i5    0  256      0

Flash_Storage
0          0
1        128
2          0
3          0
4          0
```

```
[245]: df.select_dtypes(include='number').corr()['Price']
```

```
[245]: Ram          0.743007
Weight         0.210370
Price          1.000000
Touchscreen    0.191226
Ips            0.252208
ppi           0.473487
HDD           -0.096441
SSD           0.670799
Hybrid         0.007989
Flash_Storage  -0.040511
Name: Price, dtype: float64
```

```
[246]: df.drop(columns=['Hybrid', 'Flash_Storage'],inplace=True)
```

```
[247]: df.head()
```

```
[247]: Company   TypeName  Ram      Gpu  OpSys  Weight  \
0   Apple  Ultrabook    8  Intel Iris Plus Graphics 640  macOS    1.37
1   Apple  Ultrabook    8      Intel HD Graphics 6000  macOS    1.34
2    HP    Notebook    8      Intel HD Graphics 620  No OS    1.86
3   Apple  Ultrabook   16      AMD Radeon Pro 455  macOS    1.83
```

4	Apple	Ultrabook	8	Intel Iris Plus Graphics 650	macOS	1.37
---	-------	-----------	---	------------------------------	-------	------

	Price	Touchscreen	Ips	ppi	Cpu brand	HDD	SSD
0	71378.6832	0	1	226.983005	Intel Core i5	0	128
1	47895.5232	0	0	127.677940	Intel Core i5	0	0
2	30636.0000	0	0	141.211998	Intel Core i5	0	256
3	135195.3360	0	1	220.534624	Intel Core i7	0	512
4	96095.8080	0	1	226.983005	Intel Core i5	0	256

```
[248]: df['Gpu'].value_counts()
```

```
[248]: Gpu
Intel HD Graphics 620      281
Intel HD Graphics 520      185
Intel UHD Graphics 620      68
Nvidia GeForce GTX 1050     66
Nvidia GeForce GTX 1060     48
...
AMD Radeon R7 Graphics      1
AMD Radeon R7 M360          1
Nvidia Quadro M3000M        1
Nvidia GeForce 960M         1
ARM Mali T860 MP4           1
Name: count, Length: 110, dtype: int64
```

```
[249]: df['Gpu brand'] = df['Gpu'].apply(lambda x:x.split()[0])
```

```
[250]: df.head()
```

```
[250]: Company  TypeName  Ram          Gpu  OpSys  Weight  \
0   Apple  Ultrabook    8  Intel Iris Plus Graphics 640  macOS    1.37
1   Apple  Ultrabook    8      Intel HD Graphics 6000  macOS    1.34
2    HP    Notebook    8      Intel HD Graphics 620  No OS    1.86
3   Apple  Ultrabook   16      AMD Radeon Pro 455  macOS    1.83
4   Apple  Ultrabook    8  Intel Iris Plus Graphics 650  macOS    1.37
```

	Price	Touchscreen	Ips	ppi	Cpu brand	HDD	SSD	\
0	71378.6832	0	1	226.983005	Intel Core i5	0	128	
1	47895.5232	0	0	127.677940	Intel Core i5	0	0	
2	30636.0000	0	0	141.211998	Intel Core i5	0	256	
3	135195.3360	0	1	220.534624	Intel Core i7	0	512	
4	96095.8080	0	1	226.983005	Intel Core i5	0	256	

```
Gpu brand
0   Intel
1   Intel
2   Intel
```

```
3      AMD
4      Intel
```

```
[251]: df['Gpu brand'].value_counts()
```

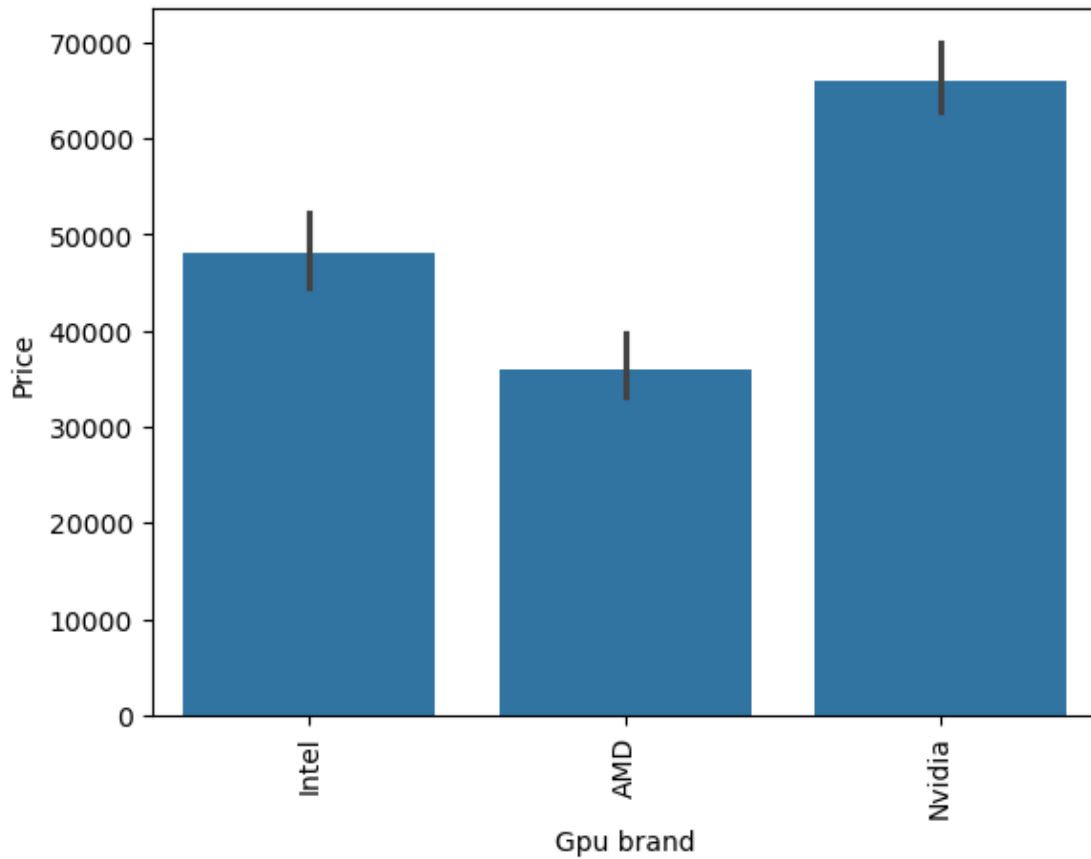
```
[251]: Gpu brand
Intel      722
Nvidia     400
AMD        180
ARM         1
Name: count, dtype: int64
```

```
[252]: df = df[df['Gpu brand'] != 'ARM']
```

```
[253]: df['Gpu brand'].value_counts()
```

```
[253]: Gpu brand
Intel      722
Nvidia     400
AMD        180
Name: count, dtype: int64
```

```
[254]: sns.barplot(x=df['Gpu brand'],y=df['Price'],estimator=np.median)
plt.xticks(rotation='vertical')
plt.show()
```

```
[255]: df.drop(columns=['Gpu'],inplace=True)
```

/tmp/ipykernel_75163/1111925144.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df.drop(columns=['Gpu'],inplace=True)
```

```
[256]: df.head()
```

```
[256]:
```

	Company	TypeName	Ram	OpSys	Weight	Price	Touchscreen	Ips	\
0	Apple	Ultrabook	8	macOS	1.37	71378.6832	0	1	
1	Apple	Ultrabook	8	macOS	1.34	47895.5232	0	0	
2	HP	Notebook	8	No OS	1.86	30636.0000	0	0	
3	Apple	Ultrabook	16	macOS	1.83	135195.3360	0	1	
4	Apple	Ultrabook	8	macOS	1.37	96095.8080	0	1	

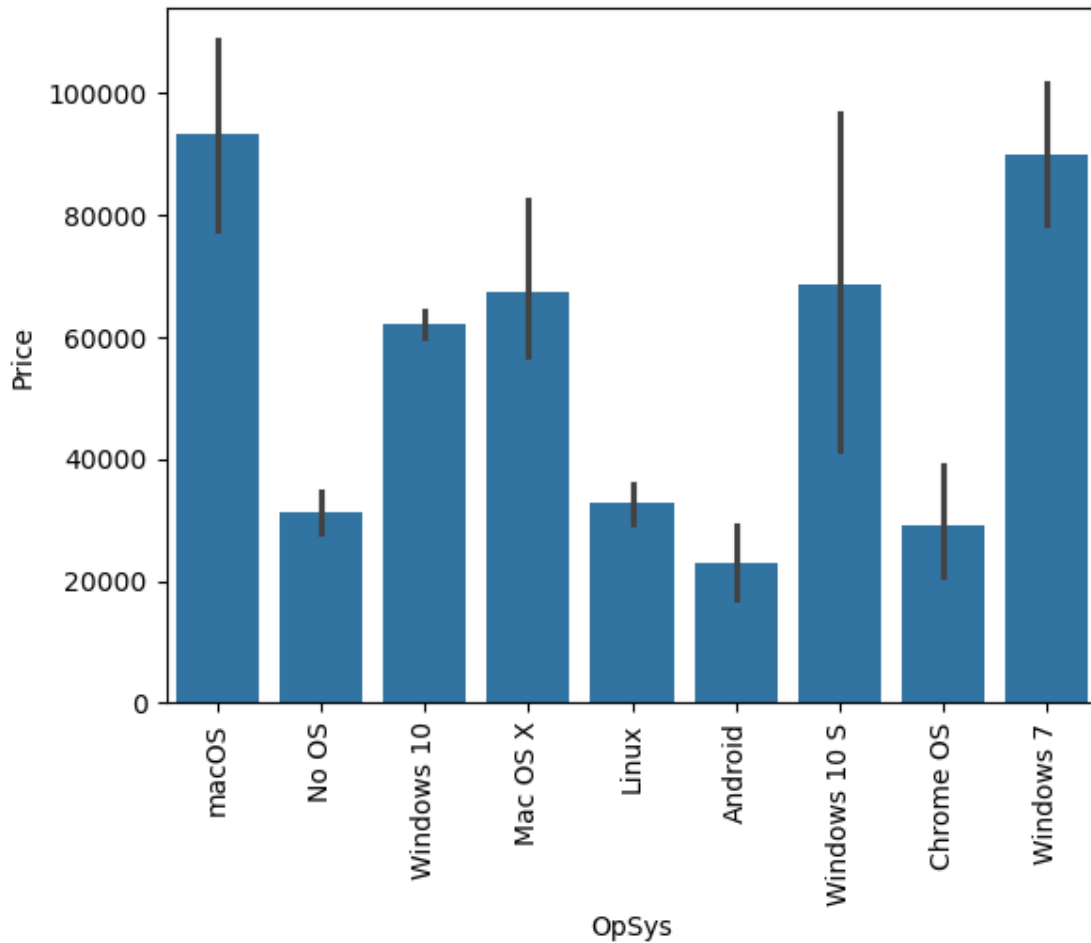
	ppi	Cpu brand	HDD	SSD	Gpu brand
0	226.983005	Intel Core i5	0	128	Intel

1	127.677940	Intel Core i5	0	0	Intel
2	141.211998	Intel Core i5	0	256	Intel
3	220.534624	Intel Core i7	0	512	AMD
4	226.983005	Intel Core i5	0	256	Intel

```
[257]: df['OpSys'].value_counts()
```

```
[257]: OpSys
Windows 10      1072
No OS           66
Linux           62
Windows 7       45
Chrome OS       26
macOS           13
Mac OS X        8
Windows 10 S    8
Android         2
Name: count, dtype: int64
```

```
[258]: sns.barplot(x=df['OpSys'],y=df['Price'])
plt.xticks(rotation='vertical')
plt.show()
```



```
[259]: def cat_os(inp):
        if inp == 'Windows 10' or inp == 'Windows 7' or inp == 'Windows 10 S':
            return 'Windows'
        elif inp == 'macOS' or inp == 'Mac OS X':
            return 'Mac'
        else:
            return 'Others/No OS/Linux'
```

```
[260]: df['os'] = df['OpSys'].apply(cat_os)
```

/tmp/ipykernel_75163/3648919379.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df['os'] = df['OpSys'].apply(cat_os)

```
[261]: df.head()
```

```
[261]:
```

	Company	TypeName	Ram	OpSys	Weight	Price	Touchscreen	Ips	\
0	Apple	Ultrabook	8	macOS	1.37	71378.6832	0	1	
1	Apple	Ultrabook	8	macOS	1.34	47895.5232	0	0	
2	HP	Notebook	8	No OS	1.86	30636.0000	0	0	
3	Apple	Ultrabook	16	macOS	1.83	135195.3360	0	1	
4	Apple	Ultrabook	8	macOS	1.37	96095.8080	0	1	

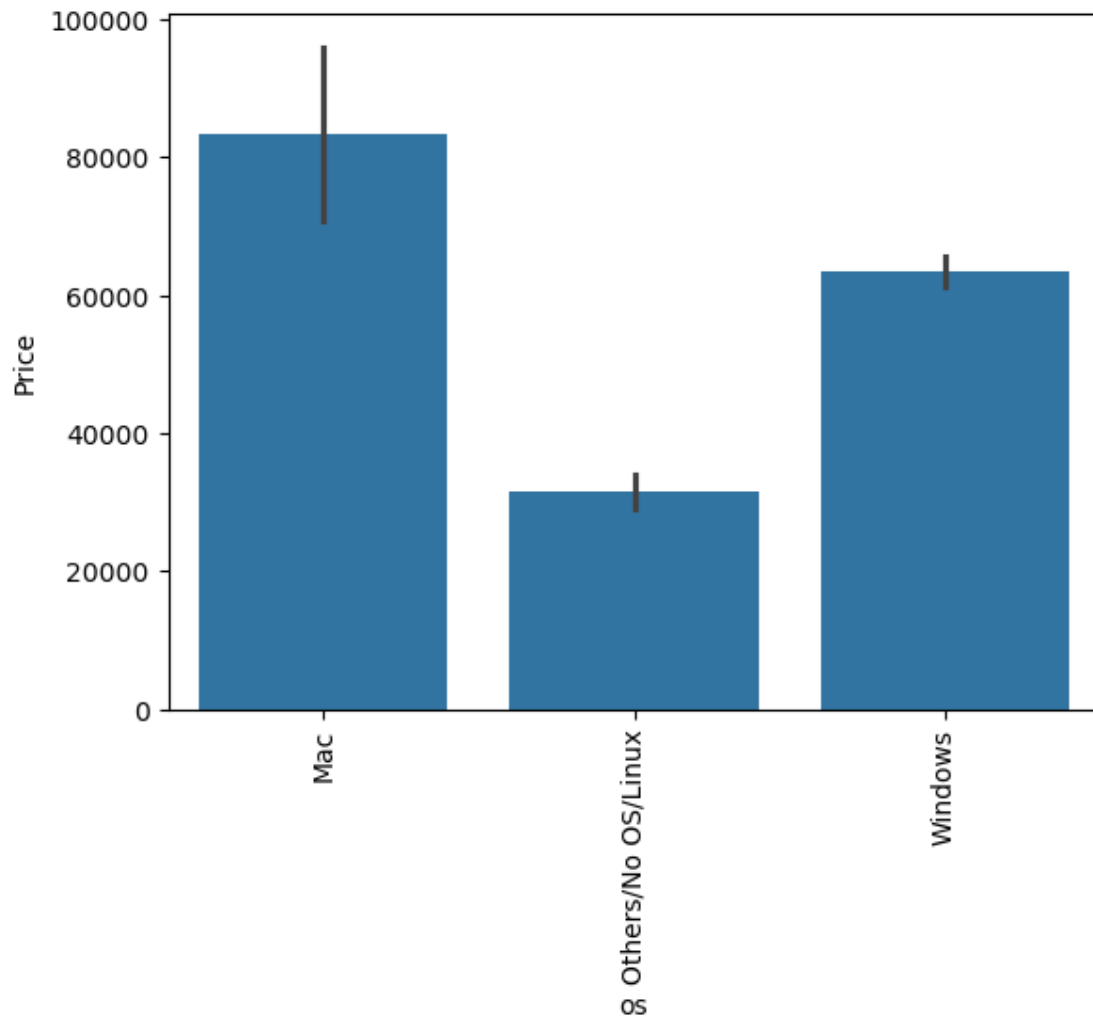
	ppi	Cpu brand	HDD	SSD	Gpu brand	os
0	226.983005	Intel Core i5	0	128	Intel	Mac
1	127.677940	Intel Core i5	0	0	Intel	Mac
2	141.211998	Intel Core i5	0	256	Intel	Others/No OS/Linux
3	220.534624	Intel Core i7	0	512	AMD	Mac
4	226.983005	Intel Core i5	0	256	Intel	Mac

```
[262]: df.drop(columns=['OpSys'],inplace=True)
```

```
/tmp/ipykernel_75163/3105339334.py:1: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy  
df.drop(columns=['OpSys'],inplace=True)
```

```
[263]: sns.barplot(x=df['os'],y=df['Price'])  
plt.xticks(rotation='vertical')  
plt.show()
```



```
[265]: sns.distplot(df['Weight'])
```

/tmp/ipykernel_75163/1125578356.py:1: UserWarning:

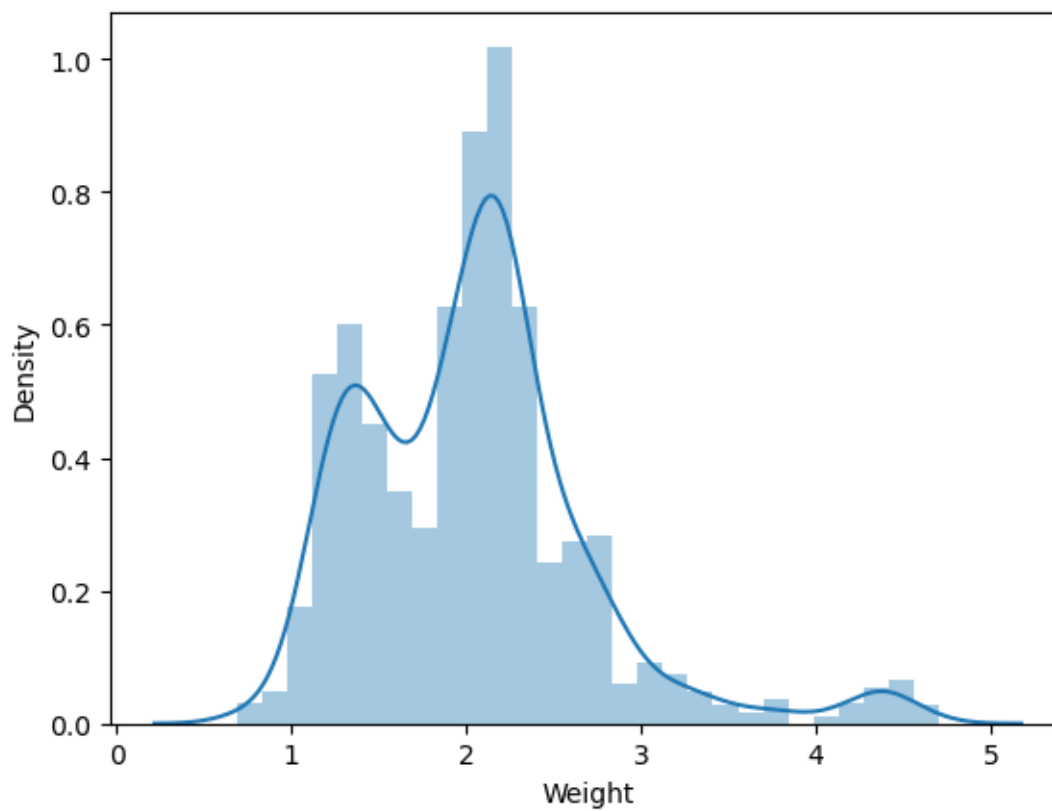
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

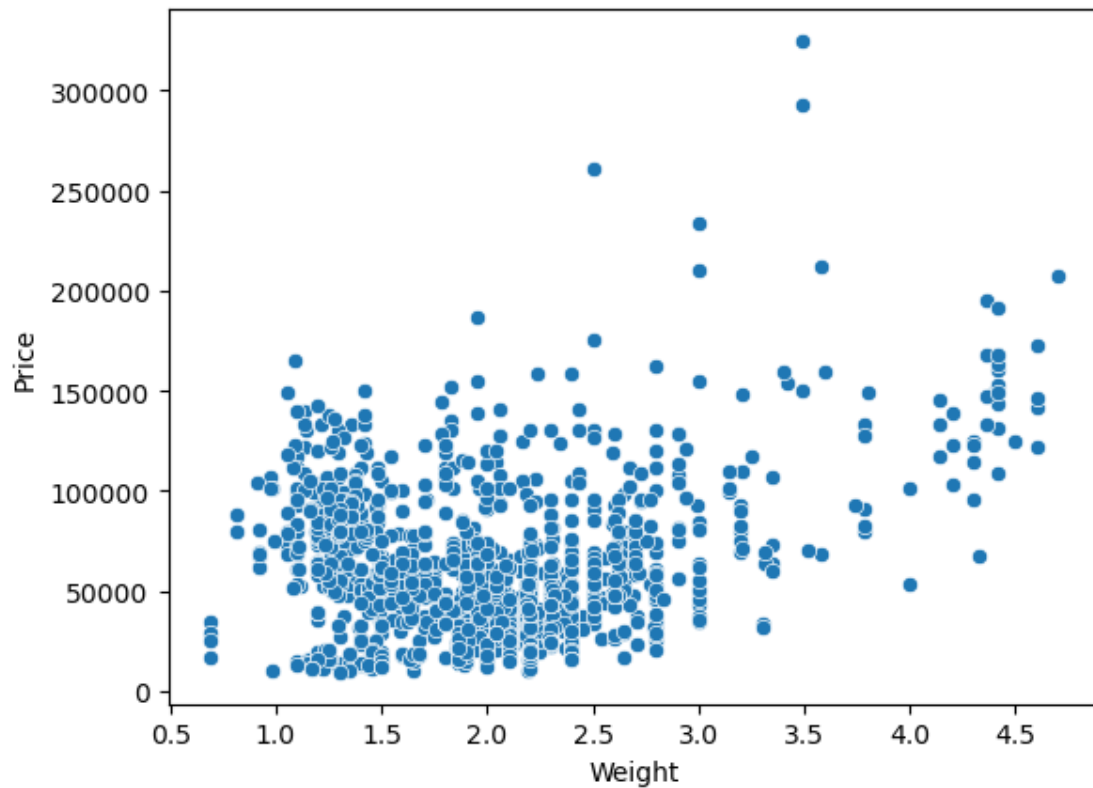
```
sns.distplot(df['Weight'])
```

```
[265]: <AxesSubplot: xlabel='Weight', ylabel='Density'>
```



```
[266]: sns.scatterplot(x=df['Weight'],y=df['Price'])
```

```
[266]: <AxesSubplot: xlabel='Weight', ylabel='Price'>
```

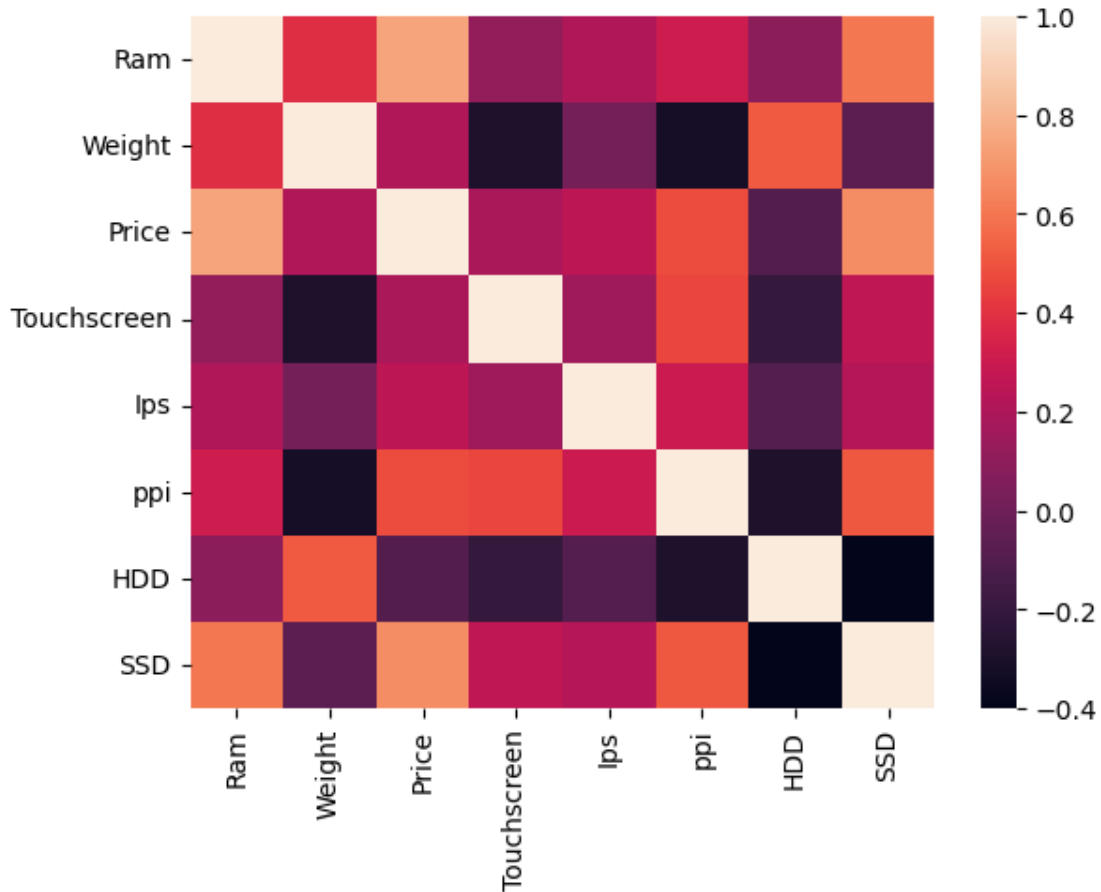


```
[267]: df.select_dtypes(include='number').corr()['Price']
```

```
[267]: Ram          0.742905
      Weight       0.209867
      Price        1.000000
      Touchscreen  0.192917
      Ips          0.253320
      ppi          0.475368
      HDD         -0.096891
      SSD          0.670660
      Name: Price, dtype: float64
```

```
[269]: sns.heatmap(df.select_dtypes(include='number').corr())
```

```
[269]: <AxesSubplot: >
```



```
[270]: sns.distplot(np.log(df['Price']))
```

/tmp/ipykernel_75163/3556049916.py:1: UserWarning:

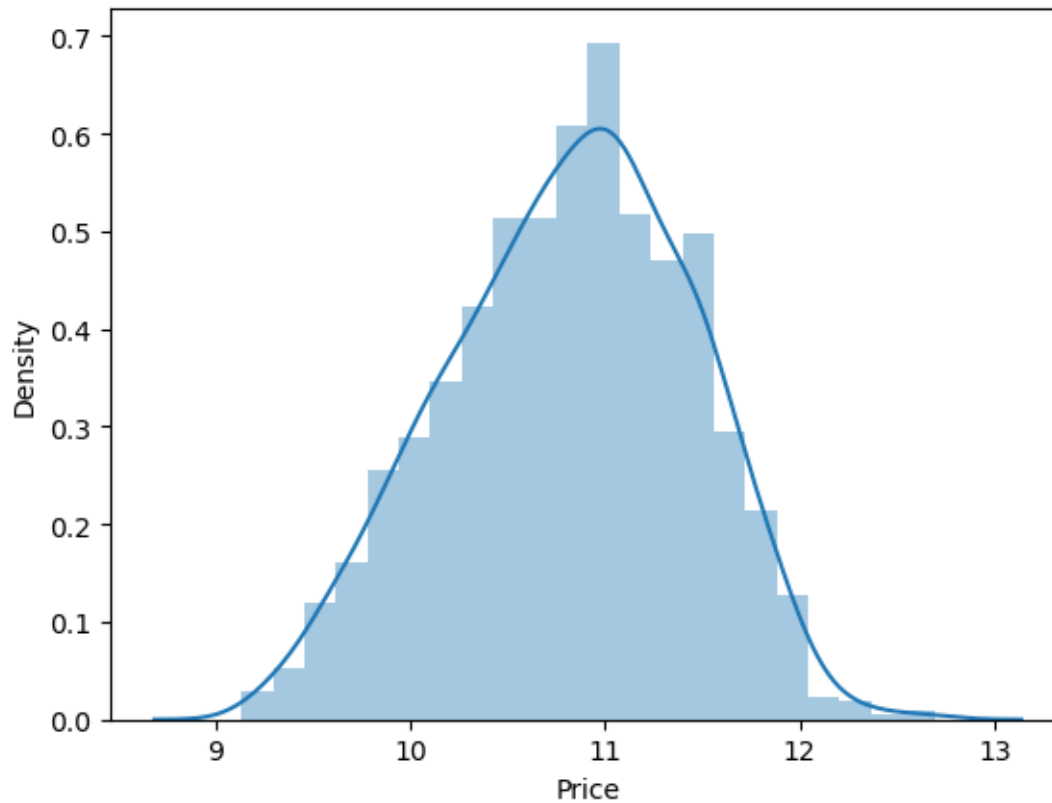
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(np.log(df['Price']))
```

```
[270]: <AxesSubplot: xlabel='Price', ylabel='Density'>
```

```
[276]: X = df.drop(columns=['Price'])
       y = np.log(df['Price'])
```

```
[305]: from sklearn.model_selection import train_test_split
       X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.
       ↪15,random_state=2)
```

```
[306]: from sklearn.compose import ColumnTransformer
       from sklearn.pipeline import Pipeline
       from sklearn.preprocessing import OneHotEncoder
       from sklearn.metrics import r2_score,mean_absolute_error
```

```
[307]: from sklearn.ensemble import RandomForestRegressor
```

```
[413]: col_transformer = ColumnTransformer(
       transformers=[
           ('onehot', OneHotEncoder(sparse_output=False, drop='first'), [0, 1, 7, ↪
       ↪10, 11])
       ],
       remainder='passthrough'
       )
```

```

rf_regressor = RandomForestRegressor(
    n_estimators=74,
    random_state=5,
    max_samples=0.8,
    max_features=1,
    max_depth=19,
)

pipe = Pipeline([
    ('preprocessor', col_transformer),
    ('regressor', rf_regressor)
])

pipe.fit(X_train, y_train)

y_pred = pipe.predict(X_test)

print('R2 Score:', r2_score(y_test, y_pred))
print('MAE:', mean_absolute_error(y_test, y_pred))

```

R2 Score: 0.9094921614316613

MAE: 0.1419151635099536

```
[414]: import pickle
```

```
[415]: pickle.dump(df, open('df.pkl', 'wb'))
pickle.dump(pipe, open('pipe.pkl', 'wb'))
```

```
[ ]:
```